Universidad Internacional de Valencia

Implementación de un sistema de detección de puntos de interés turístico mediante visión por computadora para aplicaciones de realidad aumentada

Titulación:
Master en Inteligencia Artificial
Curso académico
2025

Alumno/a: **Díaz Velasco, Rodrigo Ezequiel**

Director/a de TFM: **Yaneth Moreno Caldera**

Convocatoria:

Segunda

8 de septiembre de 2025

De:
Planeta Formación y Universidades

El verdadero viaje de descubrimiento
no consiste en buscar nuevos paisajes,
sino en mirar con nuevos ojos.

Marcel Proust

Índice general

Índice de figuras IV

Índice de tablas VI

Índice de algoritmos VII

Resumen 1

Abstract 2

1. Introducción 3
   1.1. Planteamiento del problema 3
   1.2. Justificación 4
   1.3. Objetivos 4
   1.4. Alcance 5
   1.5. Estructura del trabajo 5

2. Estado del arte 7
   2.1. Detección de objetos 7
      2.1.1. Detección en etapas 7
      2.1.2. Detección en un paso 8
   2.2. Extracción de características 11
      2.2.1. Métodos manuales 11
      2.2.2. Métodos con aprendizaje profundo 13
   2.3. Conjuntos de datos 15

3. Marco teórico 19
   3.1. Detección de objetos 19
      3.1.1. Bounding boxes 19
      3.1.2. Intersection over Union 20
      3.1.3. Non-Maximum Suppression 21
      3.1.4. ROIPool y ROIAlign 21

3.2. Extracción de características . . . 22
3.2.1. ResNet . . . 23
3.2.2. Entrenamiento CVNet-Global . . . 24
3.3. Creación del embedding . . . 26
3.3.1. GeM . . . 26
3.3.2. SuperGlobal . . . 27
3.4. Cálculo de similitud . . . 28
3.5. Aplicación Android con realidad aumentada . . . 29
3.5.1. Android Studio . . . 29
3.5.2. Open Neural Network Exchange . . . 29
3.5.3. Realidad aumentada . . . 29

4. Materiales y métodos . . . 31
4.1. Materiales . . . 31
4.1.1. Software . . . 31
4.1.2. Hardware . . . 32
4.2. Metodología . . . 32
4.2.1. Comprensión del negocio . . . 33
4.2.2. Comprensión de los datos . . . 33
4.2.3. Preparación de los datos . . . 36
4.2.4. Modelado . . . 36
4.2.5. Evaluación . . . 41
4.2.6. Despliegue . . . 41

5. Resultados . . . 43
5.1. Parámetros del sistema . . . 43
5.1.1. Evaluación cuantitativa . . . 44
5.1.2. Ejemplos cualitativos . . . 44
5.2. Construcción de la base de embeddings . . . 45
5.3. Parámetros para la detección . . . 47
5.3.1. Filtrado de detecciones . . . 48
5.4. Parámetros de *SuperGlobal* pooling . . . 49
5.5. Resumen de resultados sobre la creación de embeddings . . . 51
5.6. Parámetros de búsqueda . . . 52
5.6.1. Selección del *landmark* . . . 52
5.6.2. Refinamiento del resultado . . . 55
5.7. Resultados sobre Cáceres . . . 55

6. Conclusiones, limitaciones y perspectivas a futuro . . . 59
6.1. Conclusiones . . . 59
6.2. Limitaciones . . . 60
6.3. Perspectivas a futuro . . . 61

Lista de Acrónimos 63

A. Apéndice A 64

B. Apéndice B 65

Bibliografía 68

Índice de figuras

2.1. Evolución de R-CNN 8
2.2. Diagramas esquemáticos de SSD y YOLO 9
2.3. Verificación geométrica de características locales 12
2.4. DELG (DEep Local and Global features) 14
2.5. CVNet-Global 15
2.6. CVNet-Rerank 15
2.7. SuperGlobal 16
2.8. Muestra de imágenes de los datasets $\mathcal{R}$Oxford y $\mathcal{R}$Paris 17
2.9. Distribución geográfica del dataset Google Landmarks v2 17

3.1. Pipeline de detección de objetos con image retrieval 19
3.2. Formatos para representación de bounding boxes 20
3.3. ROIAlign: Grilla para interpolación bilineal 23
3.4. Bloque genérico de ResNet 24
3.5. Comparativas de imágenes del dataset Revisited Oxford 25

4.1. Esquema de la metodología CRISP-DM 32
4.2. Muestra de imágenes turísticas de *Open Images V7* 34
4.3. Muestra de imágenes de *Google Landmarks Dataset v2* 35
4.4. Esquema del *pipeline* completo 38

5.1. Ejemplos de resultados obtenidos sobre el dataset de Oxford con imágenes completas 45
5.2. Estrategias para la creación de embeddings 46
5.3. Rendimiento según el valor de `min_sim_db` 46
5.4. Rendimiento según el valor de `min_area` 47
5.5. Rendimiento según el valor de `score_thresh` 48
5.6. Tiempo de inferencia según el valor de `score_thresh` 49
5.7. Rendimiento según el valor de `iou_thresh`. 49
5.8. Tiempo de inferencia según el valor de `iou_thresh` 50
5.9. Rendimiento según el valor de `rgem_pr` y `rgem_size` 50

5.10. Rendimiento según el valor de `gem_p` 51
5.11. Rendimiento según el valor de `sgem_ps` 51
5.12. Relación entre precisión y cobertura 53
5.13. Precisión y cobertura en función de `min_votes` 54
5.14. Precisión y cobertura en función de `min_sim` 54
5.15. Ejemplo de eliminación de detecciones redundantes 55
5.16. Ejemplo de agrupación de detecciones del mismo *landmark* 56
5.17. Ejemplos con detecciones correctas sobre Cáceres 57
5.18. Ejemplo de detección parcial en sitios vecinos. 57
5.19. Ejemplos de casos no detectados sobre Cáceres 58

Índice de tablas

4.1. Anotaciones en *Open Images V7* 34
4.2. Anotaciones en *Google Landmarks Dataset v2* 35
4.3. Anotaciones en *Revisited Oxford and Paris* 36

5.1. Lista de parámetros del modelo 43
5.2. Valores *default* de los parámetros del modelo 44
5.3. Resultados en *Revisited Oxford and Paris* 45
5.4. Subconjunto de clases de *Open Images V7* 47
5.5. Parámetros optimizados para la creación de *embeddings* 52
5.6. Comparativa de resultados en *Revisited Oxford and Paris* 52

Índice de algoritmos

1. *Non-Maximum Suppression (NMS)* 21
2. *RoIPool* 22
3. *RoIAlign* 22

Resumen

El presente trabajo aborda el diseño e implementación de un sistema de visión por computadora orientado al turismo, con el objetivo de enriquecer la experiencia de los visitantes mediante tecnologías de inteligencia artificial y realidad aumentada. La propuesta surge ante la necesidad de ofrecer soluciones innovadoras que permitan democratizar el acceso al patrimonio cultural y natural, ampliando las posibilidades de destinos con menor visibilidad turística y contribuyendo a un desarrollo más equilibrado del sector.

Para ello, se ha realizado un análisis exhaustivo del estado del arte en detección de objetos y técnicas de *image retrieval*, revisando tanto los métodos clásicos como las aproximaciones modernas basadas en aprendizaje profundo. Este estudio permitió fundamentar el diseño de un *pipeline* capaz de identificar puntos de interés en imágenes capturadas por dispositivos móviles, robusto frente a variaciones en iluminación, ángulo u oclusiones.

El desarrollo se estructuró en varias etapas siguiendo la metodología CRISP-DM: preparación de datos, modelado, evaluación y despliegue. Se utilizaron datasets públicos de referencia (*Open Images V7*, *Google Landmarks v2* y *Revisited Oxford and Paris*), además de un conjunto propio de imágenes de la ciudad de Cáceres para validar la aplicación en escenarios reales. El sistema final integra detección de objetos, extracción de características visuales y búsqueda por similitud en una base de *embeddings*, todo ello exportado al formato ONNX para garantizar portabilidad y eficiencia en dispositivos móviles.

Los resultados obtenidos muestran un rendimiento competitivo en *benchmarks* internacionales y un funcionamiento satisfactorio en pruebas de campo sobre Cáceres, donde el sistema fue capaz de reconocer múltiples puntos de interés en condiciones reales. La integración en una aplicación Android con soporte de realidad aumentada demuestra su aplicabilidad práctica, ofreciendo al usuario una experiencia inmersiva y dinámica. Finalmente, se destacan las limitaciones detectadas y se plantean líneas de mejora futura, consolidando así una contribución innovadora en la intersección entre turismo, inteligencia artificial y tecnologías inmersivas.

Abstract

This work addresses the design and implementation of a computer vision system applied to the tourism sector, with the aim of enhancing visitor experiences through artificial intelligence and augmented reality technologies. The proposal arises from the need to provide innovative solutions that democratize access to cultural and natural heritage, expanding the visibility of less-known destinations and contributing to a more balanced development of the sector.

To this end, an in-depth analysis of the state of the art in object detection and image retrieval techniques was carried out, reviewing both classical methods and modern deep learning approaches. This study served as the basis for designing a pipeline capable of identifying landmarks in images captured by mobile devices, while being robust to variations in lighting, viewpoint, or occlusions.

The development followed the CRISP-DM methodology, covering data preparation, modeling, evaluation, and deployment. Public benchmark datasets were used (Open Images V7, Google Landmarks v2, and Revisited Oxford and Paris), as well as a custom dataset of the city of Cáceres to validate the system in real-world scenarios. The final solution integrates object detection, visual feature extraction, and similarity-based search in an embedding database, all exported to ONNX format to ensure portability and efficiency on mobile devices.

The results demonstrate competitive performance on international benchmarks and satisfactory operation in field tests carried out in Cáceres, where the system successfully recognized multiple landmarks in real conditions. Its integration into an Android application with augmented reality support highlights its practical applicability, offering users an immersive and dynamic experience. Finally, the identified limitations are discussed, and future research directions are outlined, consolidating an innovative contribution at the intersection of tourism, artificial intelligence, and immersive technologies.

Introducción 1

El presente trabajo busca desarrollar una solución tecnológica que transforme la forma en que los visitantes interactúan con el patrimonio cultural y natural, especialmente en destinos con baja visibilidad turística. A través de la implementación de un sistema móvil de visión por computadora integrado con realidad aumentada, se espera democratizar el acceso a la información turística, haciendo posible que cualquier lugar, sin importar su popularidad o recursos, pueda ofrecer experiencias inmersivas y personalizadas. En un futuro cercano, este tipo de soluciones también podrán integrarse directamente en dispositivos de realidad aumentada, como gafas inteligentes, permitiendo una experiencia aún más fluida.

La propuesta tiene el potencial de ampliar la distribución geográfica del turismo, mitigar la sobrecarga en destinos tradicionales y reducir las barreras de entrada para nuevas regiones. Asimismo, al facilitar el reconocimiento de nuevos sitios sin necesidad de reentrenamiento, se promueve un sistema dinámico y escalable que acompaña el crecimiento continuo del sector. Esta tecnología no solo optimiza la experiencia del usuario final, sino que también representa un recurso estratégico para mejorar la competitividad de destinos y operadores turísticos.

1.1. Planteamiento del problema

En la actualidad, el turismo tiende a concentrarse en regiones consolidadas, donde el alto volumen de visitantes permite sostener infraestructuras costosas, como la presencia de guías turísticos que facilitan el acceso al patrimonio cultural y natural. En contraste, los destinos menos conocidos no cuentan con los recursos necesarios para ofrecer este tipo de servicios, lo que limita significativamente la experiencia de los visitantes y su capacidad para descubrir los atractivos locales.

Algunos puntos de interés, como museos, palacios o ciudades con una fuerte identidad turística, disponen de sus propias aplicaciones móviles que actúan como guías digitales. No obstante, este tipo de soluciones presentan una serie de limitaciones significativas, entre ellas:

- Suelen implicar un coste de desarrollo elevado, lo que las hace inviables para destinos con menor afluencia de turistas o recursos limitados.
- No son escalables a una gran cantidad de ubicaciones.

- En muchos casos, no integran tecnologías avanzadas como la inteligencia artificial para mejorar la experiencia del usuario.

Como consecuencia, una gran parte del patrimonio cultural y natural en zonas menos populares permanece invisible para los visitantes. Esta situación no solo reduce la diversidad de la oferta turística, sino que también contribuye a un círculo vicioso: los turistas se concentran en un número reducido de destinos, lo que intensifica los impactos sociales y ambientales en esos lugares y acentúa el desequilibrio en la distribución de visitantes.

1.2. Justificación

Este proyecto surge como respuesta a la necesidad de brindar soluciones tecnológicas accesibles y escalables que mejoren la experiencia del visitante en destinos turísticos con recursos limitados. A través del uso de redes neuronales y visión por computadora, se busca democratizar el acceso a información contextualizada sin depender de guías presenciales o aplicaciones específicas.

Al evitar la necesidad de reentrenamiento para incorporar nuevos lugares, el sistema propuesto ofrece una alternativa dinámica y eficiente, alineada con las tendencias emergentes en inteligencia artificial y realidad aumentada.

1.3. Objetivos

El objetivo general de este proyecto es implementar un sistema móvil de visión por computadora capaz de detectar puntos de interés turístico y operar en entornos de realidad aumentada, permitiendo la incorporación de nuevos lugares sin requerir reentrenamiento del modelo.

Para alcanzar el objetivo general mencionado, deben cumplirse una serie de objetivos específicos detallados a continuación:

1. Analizar el estado del arte en detección de objetos y técnicas de *image retrieval*.

2. Implementar una arquitectura de redes neuronales profundas robusta frente a variaciones visuales, oclusiones y elementos distractores.

3. Diseñar una base de datos de puntos de interés turístico estructurada mediante vectores de características visuales.

4. Integrar el sistema con un *framework* de realidad aumentada en dispositivos móviles.

5. Evaluar el desempeño del sistema utilizando métricas de precisión y tiempo de inferencia.

1.4. Alcance

El presente trabajo se centrará en el desarrollo de un sistema capaz de identificar puntos de interés turístico presentes en imágenes capturadas desde un dispositivo móvil. Para ello, se construirá un *pipeline* de visión por computadora que, ante una imagen de entrada, será capaz de detectar objetos relevantes, generar descriptores robustos que capturen sus características invariantes y realizar una búsqueda por similitud eficiente sobre una base de datos previamente construida.

Este sistema se diseñará con el objetivo de operar en tiempo real, de forma que pueda integrarse en entornos interactivos sin comprometer la experiencia del usuario. Además, se garantizará su compatibilidad con el formato estándar ONNX, lo que permitirá su despliegue en una amplia variedad de plataformas y dispositivos.

El *pipeline* será incorporado en una aplicación para dispositivos Android. Esta aplicación utilizará la cámara del móvil para capturar imágenes, enviarlas al modelo desarrollado y superponer sobre la imagen original los puntos de interés turístico que hayan sido detectados. La visualización de estos elementos se realizará mediante un entorno de realidad aumentada, permitiendo al usuario explorar e identificar sitios relevantes directamente sobre la escena capturada en tiempo real.

1.5. Estructura del trabajo

Este documento se encuentra organizado en los siguientes capítulos. El capítulo 2 está dedicado al estado del arte, donde se analiza la evolución de los métodos de detección de objetos (sección 2.1), las técnicas de extracción de características invariantes en imágenes de lugares (sección 2.2), y los principales conjuntos de datos empleados en este campo (sección 2.3).

En el capítulo 3 se introducen los conceptos teóricos fundamentales que sustentan la implementación del sistema propuesto. El capítulo 4 describe los materiales utilizados y detalla, paso a paso, la metodología seguida para el desarrollo del proyecto, incluyendo la implementación completa del *pipeline*.

La selección de parámetros del sistema, junto con la presentación y análisis de los resultados obtenidos, se abordan en el capítulo 5. Finalmente, el capítulo 6 expone las conclusiones generales del trabajo (sección 6.1), discute sus principales limitaciones (sección 6.2) y plantea posibles líneas de trabajo futuro (sección 6.3).

Estado del arte

2

La detección de puntos de interés turísticos se aborda como un problema de *Visual Place Recognition* (VPR), disciplina con aplicaciones también en robótica para el posicionamiento autónomo. Su objetivo es determinar la ubicación geográfica de una imagen comparándola con una base de datos de vistas de referencia asociadas a sitios conocidos. A diferencia de la búsqueda de similitud visual convencional, aquí no basta con encontrar imágenes parecidas; se trata de recuperar aquellas que representen exactamente el mismo lugar, pese a cambios de iluminación, ángulo de toma o condiciones atmosféricas, motivo por el cual se le denomina *instance image retrieval*.

2.1. Detección de objetos

La detección de objetos es una disciplina fundamental de la visión por computadora que combina la localización de regiones de interés con la clasificación de cada una de ellas. En la fase de localización, el algoritmo propone recuadros delimitadores que encierran posibles objetos. A continuación, el componente de clasificación asigna a cada recuadro la etiqueta de la categoría correspondiente. De este modo, es posible identificar y cuantificar múltiples objetos de distintas clases dentro de una misma imagen.

2.1.1. Detección en etapas

En 2014, Girshick et al. (2014) introdujeron *Region-based CNN* (R-CNN), un método de detección de objetos basado en tres pasos independientes: extracción de propuestas de regiones, extracción de características de cada región con una CNN y clasificación. Al año siguiente, Girshick (2015) optimizó este flujo extrayendo las características sobre la imagen completa una sola vez y proyectando las regiones propuestas directamente sobre el mapa de características, dando lugar a *Fast* R-CNN. Aunque redujo significativamente el tiempo de inferencia, seguía dependiendo de un generador externo de propuestas, lo que se convirtió en el principal cuello de botella. En 2016, Ren et al. (2016) integraron la *Region Proposal Network* (RPN) en la red, unificando generación de regiones y detección para crear *Faster* R-CNN. Finalmente, He et al. (2018) presentaron *Mask* R-CNN, que incorporó una rama de segmentación de máscaras y sustituyó *RoIPool* por *RoIAlign*, mejorando la precisión del alineamiento espacial de las características extraídas. La Fig. 2.1 muestra los diagramas esquemáticos de las arquitecturas R-CNN.

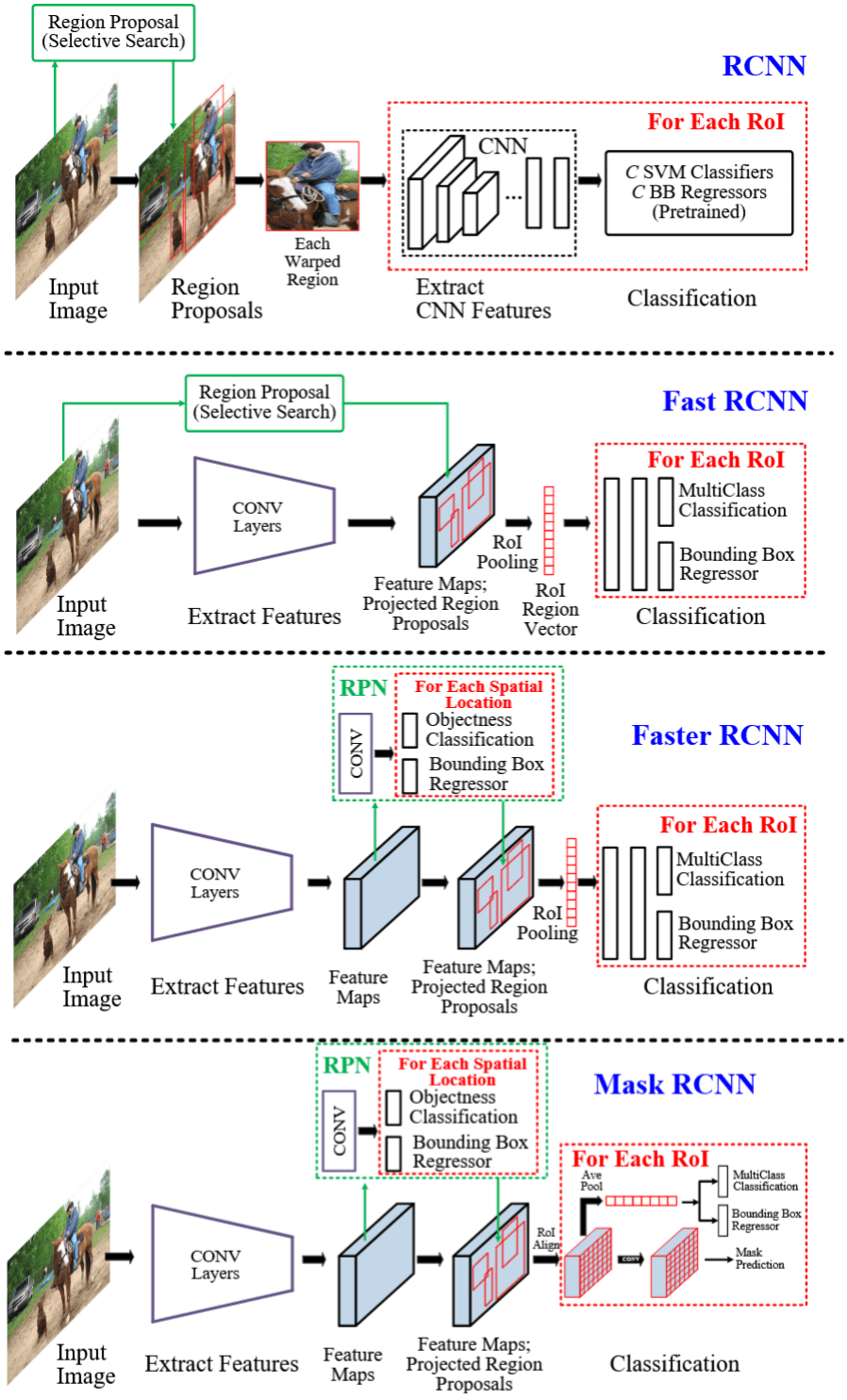


Figura 2.1: *Evolución de R-CNN. Fuente: Liu et al. (2019)*

2.1.2. Detección en un paso

Los métodos basados en regiones descritos anteriormente requieren un esquema de dos fases, lo cual resulta computacionalmente costoso, especialmente en dispositivos móviles

con recursos de cómputo y almacenamiento limitados. Además, suelen ser demasiado lentos para aplicaciones que demandan procesamiento en tiempo real.

En Liu et al. (2016) se presentó el *Single Shot Detector* (SSD), un enfoque unificado de una sola etapa que prescinde de la generación de propuestas externa. En la Fig. 2.2 se muestra es diagrama esquemático de esta arquitectura. SSD emplea una red base —por ejemplo VGG— seguida de múltiples capas convolucionales que reducen progresivamente la resolución de los mapas de activación. Cada capa predice un conjunto fijo de cajas delimitadoras (*anchors*) con diferentes escalas y proporciones, y asigna a cada una su puntuación de clasificación. Finalmente, se aplica *Non-Maximum Suppression* (NMS) para filtrar las detecciones redundantes, logrando un balance eficiente entre velocidad y precisión, adecuado para aplicaciones en tiempo real y dispositivos con recursos limitados.

Por otro lado, Redmon et al. (2015) introdujeron *You Only Look Once* (YOLO), un enfoque de una sola etapa que plantea la detección de objetos como un problema de regresión: una única red procesa la imagen completa dividida en una cuadrícula y predice simultáneamente las cajas delimitadoras, las clases y la confianza de cada detección. Este diseño unificado, esquematizado en la Fig. 2.2, permitió alcanzar velocidades de hasta 45 FPS, si bien presentaba dificultades para localizar objetos pequeños, especialmente cuando aparecían en grupo.

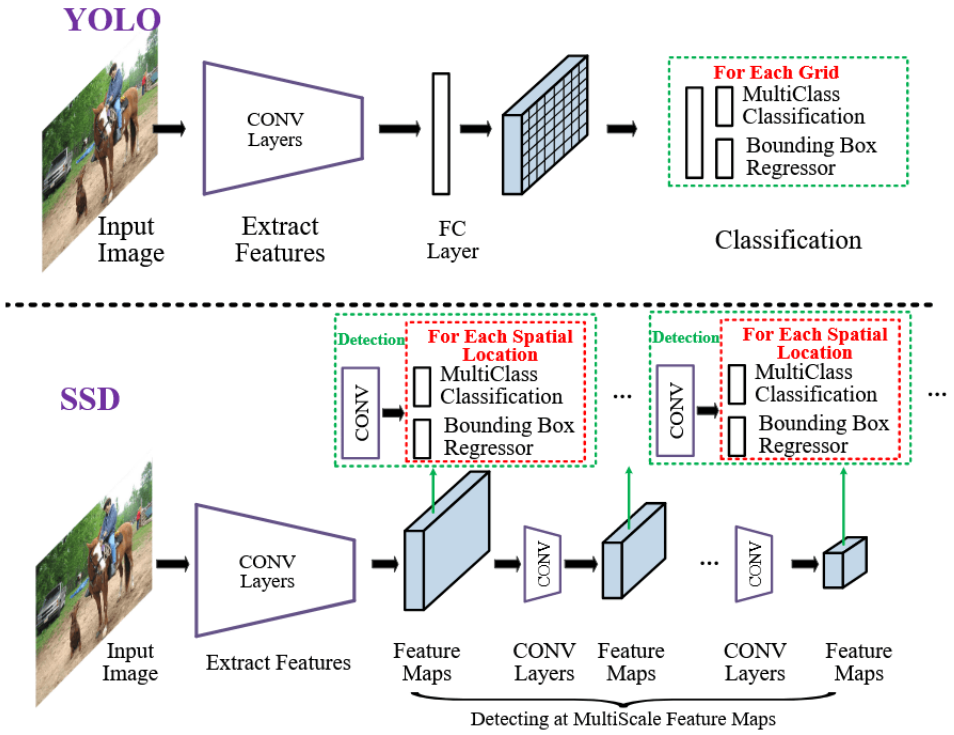


Figura 2.2: *Diagrama esquemático de SSD (abajo) y YOLO (arriba). Fuente: Liu et al. (2019)*

YOLO evolucionó en versiones sucesivas, incorporando mejoras en la arquitectura, el manejo de escalas y la precisión en la detección de objetos de distinto tamaño. Estas mejoras se describen a continuación:

- YOLOv2 - Redmon y Farhadi (2016): Se agregó *Batch Normalization* para mejorar el entrenamiento y se incorporaron *anchor boxes* para realizar varias predicciones con diferentes relaciones de aspecto sobre cada celda, superando la limitación de detección de objetos de distintos tamaños.
- YOLOv3 - Redmon y Farhadi (2018): introdujo una estructura inspirada en *Feature Pyramid Networks*, utilizando salidas de múltiples capas para mejorar la detección a diferentes escalas, y sustituyó el *backbone* por Darknet-53.
- YOLOv4 - Bochkovskiy et al. (2020): incorporó técnicas avanzadas de aumento de datos y adoptó CSPDarknet53 como *backbone*, optimizando el rendimiento y la velocidad.
- YOLOv5: lanzado por Ultralytics en 2020 con implementación en PyTorch, mejoró la facilidad de uso, integración y despliegue, incluyendo exportación a ONNX y Core ML para aplicaciones en dispositivos móviles.
- YOLOv6 - Li et al. (2022): introdujo la variante YOLOv6-Nano, diseñada para entornos con recursos limitados, facilitando su uso en dispositivos móviles e IoT.
- YOLOv7 - Wang et al. (2022): mejoró la supresión de *Non-Maximum Suppression* (NMS) y añadió tareas avanzadas como estimación de pose.
- YOLOv8 - Yaseen (2024): incorporó *Cross-Stage Partial Networks* (CSPN) para reducir redundancias entre capas, mejorando la detección de objetos pequeños y reduciendo el tiempo de inferencia, y extendió su aplicabilidad a múltiples tareas de visión por computadora.
- YOLOv9 - Wang et al. (2024b): integró *Programmable Gradient Information* (PGI) y *Generalized Efficient Layer Aggregation Network* (GELAN) para mejorar la eficiencia y adaptabilidad del modelo.
- YOLOv10 - Wang et al. (2024a): eliminó la necesidad de *Non-Maximum Suppression* (NMS) al incorporar un mecanismo interno de filtrado de detecciones redundantes.
- YOLOv11 - Khanam y Hussain (2024): la versión más reciente de Ultralytics, que alcanza el estado del arte en detección de objetos, segmentación, estimación de pose, seguimiento y clasificación.

Recientemente, Zhao et al. (2024) introdujeron *Real-Time DEtection TRansformer* (RT-DETR) como una alternativa basada en arquitecturas *Transformer* que busca superar a YOLO en aplicaciones de detección en tiempo real, combinando un módulo de atención global con mecanismos optimizados de predicción de cajas.

Finalmente, YOLOv12 de Tian et al. (2025) representa un cambio de paradigma al incorporar completamente mecanismos de atención en su arquitectura, sustituyendo el diseño tradicional de YOLO por capas centradas en atención para mejorar la captura de relaciones espaciales y de contexto, manteniendo el enfoque de una sola etapa y optimizando tanto el rendimiento como la velocidad de inferencia.

2.2. Extracción de características

El sistema ha de extraer descriptores invariantes que capturen la esencia estructural de la escena, descartando dinámicas efímeras como peatones o vehículos. Asimismo, debe mantener su fiabilidad ante variaciones de punto de vista, cambios de escala y diferencias de iluminación, garantizando una representación coherente del mismo lugar en condiciones diversas.

2.2.1. Métodos manuales

Los métodos iniciales de *Visual Place Recognition* (VPR) se basaban en la extracción manual de características visuales —bordes, esquinas, puntos clave— y su comparación mediante descriptores diseñados a mano. Aunque han quedado superados por los enfoques de *deep learning*, muchos de sus principios fundamentales (como la noción de invariancia a rotaciones o cambios de escala) siguen vigentes. Por ello, es importante repasar su evolución para comprender cómo los avances en arquitecturas y entrenamiento automático han potenciado la precisión y robustez de la recuperación de instancias.

2.2.1.1. Características globales

En las primeras aproximaciones a VPR, los algoritmos dependían de descriptores calculados sobre la totalidad de la imagen. Estas técnicas empleaban representaciones como histogramas de color propuesto por Ulrich y Nourbakhsh (2000), descriptores derivados de análisis de componentes principales (PCA) implementado por Kröse et al. (2001) o la combinación de detectores de bordes y esquinas para generar huellas digitales (*fingerprints*) de cada escena como el de Lamon et al. (2001). Uno de los enfoques más influyentes fue el descriptor Gist presentado por Oliva y Torralba (2006), que resume la estructura espacial de la imagen en un vector de baja dimensionalidad.

Aunque los métodos de características globales suelen ser robustos ante cambios de iluminación —permitiendo reconocer lugares en distintos momentos del día, condiciones meteorológicas o estaciones— presentan dificultades cuando la posición de la cámara varía significativamente. En esas situaciones, la apariencia global de la escena puede cambiar drásticamente, dificultando la coincidencia desde ángulos diferentes. Estas limitaciones impulsaron el desarrollo de algoritmos basados en características locales, capaces de capturar detalles invariantes y mejorar la robustez frente a variaciones de punto de vista.

2.2.1.2. Características locales

En Sivic y Zisserman (2003) se introdujo el concepto de *Bag of Words* (BoW) procedente del procesamiento de texto para VPR. La idea consiste en crear un vocabulario visual agrupando descriptores locales mediante técnicas de *clustering* (p. ej., k-Means) y representar cada imagen como un histograma de palabras visuales, lo cual facilita la comparación entre escenas.

Más adelante, Lowe (2004) presentó *Scale Invariant Feature Transform* (SIFT), un método para extraer puntos clave y descriptores que son invariantes a cambios de escala, rotación y en parte a variaciones de iluminación y punto de vista. Más adelante, *Speeded Up Robust Features* (SURF) introducido por Bay et al. (2006) surgió como una alternativa optimizada para acelerar el cálculo de descriptores, conservando gran parte de la robustez de SIFT.

Los métodos basados en BoW carecen de información sobre la estructura geométrica del lugar que describen, lo que puede limitar su capacidad para distinguir escenas con configuraciones espaciales similares. Incorporar información geométrica —por ejemplo, mediante correspondencias espaciales de puntos clave o modelos de geometría múltiple vista— ha demostrado aumentar significativamente la robustez del reconocimiento de lugares. Estudios como los de Valgren y Lilienthal (2010), Cummins y Newman (2011) y Cadena et al. (2012) evidencian mejoras al fusionar descriptores visuales con información geométrica de la escena.

Una vez extraídos los descriptores locales de la imagen de consulta, se realiza una búsqueda en la base de datos para identificar las imágenes con mayor número de coincidencias. Este enfoque asume independencia entre los descriptores, una suposición que falla en entornos urbanos con estructuras repetitivas (p. ej., fachadas con múltiples ventanas). Para mitigar esta limitación, Torii et al. (2015) propusieron un método que incorpora la disposición espacial de los puntos clave, aumentando la capacidad de discriminación y mejorando el rendimiento en escenarios con alta repetitividad arquitectónica.

Los enfoques basados en emparejamiento de características locales extraen cientos de descriptores por imagen, los cuales deben cotejarse con cada descriptor de la base de datos, resultando en un proceso computacionalmente costoso y lento. Para optimizarlo, Turcot y Lowe (2009) introdujeron un paso de verificación geométrica mediante el método *Random Sample Consensus* (RANSAC) de Martin A. Fischler et al. (1981), conservando únicamente aquellos pares de puntos clave que cumplen un modelo geométrico consistente y descartando el resto como se ve en la Fig. 2.3.



(a) All image features (b) Useful image features

Figura 2.3: *Características locales detectadas **(a)** y luego de realizar la verificación geométrica **(b)**. Fuente: Turcot y Lowe (2009)*

2.2.2. Métodos con aprendizaje profundo

En 2014, Chen et al. (2014) introdujeron el primer método de reconocimiento visual de lugares basado en redes neuronales convolucionales, empleando la arquitectura preentrenada *Overfeat* (Sermanet et al. (2014)) como extractor de características. Un año después, Sünderhauf et al. (2015) determinaron que los mapas de activación de capas intermedias de las CNN ofrecen descriptores más robustos frente a variaciones de iluminación y condiciones ambientales, mientras que las capas finales proporcionan mayor invarianza a cambios de perspectiva.

Un año más tarde Hou et al. (2015) realizaron una comparación sistemática entre los enfoques manuales predominantes (GIST, BoVW, Fisher Vectors y VLAD) y los métodos basados en CNN. Concluyeron que las redes convolucionales superan a los métodos tradicionales tanto en precisión como en velocidad: ofrecen mejoras significativas en exactitud y son una orden de magnitud más rápidas en CPU y dos órdenes más rápidas en GPU. A partir de estos resultados, los enfoques basados en CNN se consolidaron como el nuevo estado del arte en reconocimiento visual de lugares.

2.2.2.1. Representaciones compactas

Babenko y Lempitsky (2015) presentaron *Sum-Pooled Convolutional features* (SPoC), basado en *sum-pooling*. Tolias et al. (2016) introdujeron *Maximum Activations of Convolutions* (MAC), aplicando *max-pooling* sobre mapas de activación para generar una representación global compacta, junto con su extensión regional R-MAC. Kalantidis et al. (2016) propusieron *Cross-Dimensional Weighting* (CroW), un mecanismo para ponderar mapas de activación antes del *pooling*. Arandjelović et al. (2016) diseñaron NetVLAD, una arquitectura CNN que emula el *Vector of Locally Aggregated Descriptors* (VLAD) para reconocimiento de lugares. Radenović et al. (2018b) generalizaron técnicas de *pooling* con *Generalized-Mean pooling* (GeM), para obtener descriptores más flexibles.

2.2.2.2. Refinamiento con características locales

Noh et al. (2018) presentaron *DEep Local Feature* (DELF), que incorpora un mecanismo de atención para identificar puntos de interés discriminativos antes de extraer descriptores locales, mejorando la calidad de las correspondencias y la eficiencia del proceso de búsqueda.

Buscar exhaustivamente en extensas bases de datos utilizando exclusivamente descriptores locales resulta ineficiente y demasiado lento para aplicaciones en tiempo real. Para mitigar este problema, Sarlin et al. (2019) propusieron un *pipeline* de *coarse-to-fine* que primero aplica una búsqueda rápida con características globales para filtrar un subconjunto de candidatos, y luego realiza un refinamiento local sobre esas instancias, reduciendo notablemente los falsos positivos.

Siguiendo esa idea, Cao et al. (2020) extendieron DELF para construir *DEep Local and Global features* (DELG), un enfoque unificado que entrena de forma *end-to-end* una arquitectura única para generar simultáneamente un descriptor global y descriptores locales como

puede observarse en la Fig. 2.4. DELG emplea tres funciones de pérdida: *ArcFace* para el descriptor global, atención supervisada (*Attention Loss*) para el módulo de atención, y el error de reconstrucción de un *autoencoder* para optimizar los descriptores locales. La búsqueda masiva se realiza inicialmente con el descriptor global, seguida de un *reranking* mediante los descriptores locales y una verificación geométrica (p. ej., RANSAC).

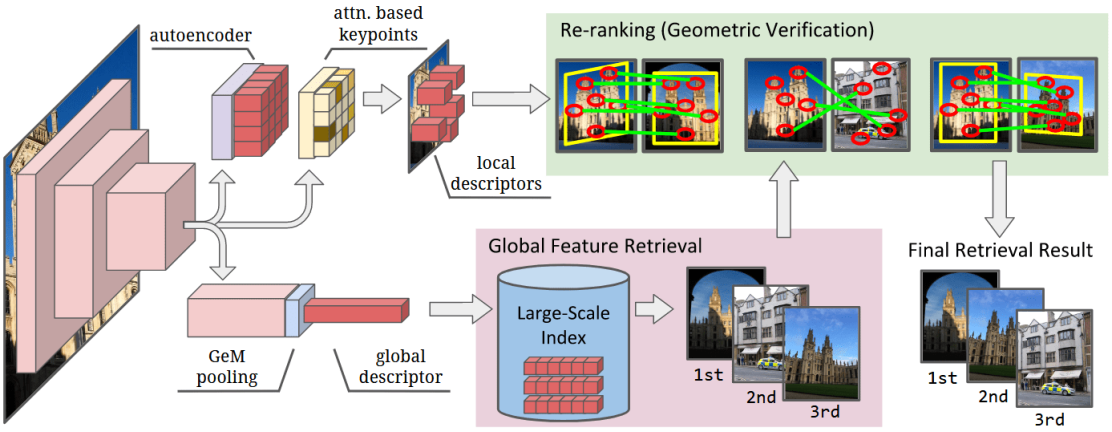


Figura 2.4: *DELG (DEep Local and Global features). Fuente: Cao et al. (2020)*

Taira et al. (2021) emplearon las activaciones de las capas finales de una CNN para la etapa global y extrajeron descriptores locales de capas intermedias para la fase de refinamiento, interpretando los mapas de activación intermedios como conjuntos de descriptores locales en una cuadrícula 2D y logrando mayor eficacia y robustez en entornos complejos.

Lee et al. (2022) propusieron *Correlation Verification Networks* (CVNet), dividida en dos componentes principales. Primero, CVNet-Global emplea una ResNet (He et al. (2015)), como extractor de características, sustituyendo el *pooling* convencional por un GeM con parámetro entrenable y optimizada mediante una combinación de pérdida de clasificación y pérdida contrastiva. A continuación, CVNet-Rerank refina las similitudes a través de capas convolucionales 4D profundas que comprimen progresivamente la correlación de características densas en una medida de similitud entre imágenes, aprendiendo patrones geométricos diversificados de distintos pares de imágenes. Para facilitar el emparejamiento a distintas escalas, construye pirámides de características y calcula correlaciones *cross-scale* en una sola inferencia, evitando el costo de realizar múltiples escalas por separado. En la Fig. 2.5 puede observarse la estructura de CVNet-Global y en la Fig. 2.6 el de CVNet-Rerank.

Recientemente, Shao et al. (2023) presentaron *SuperGlobal*, un innovador esquema de *pooling* basado en GeM con tres módulos diferenciados:

- GeM+: una versión optimizada de GeM que ajusta dinámicamente sus parámetros durante el entrenamiento.
- *Regional*-GeM: agrega de forma selectiva información de subregiones de la imagen para capturar detalles locales dentro de la representación global.

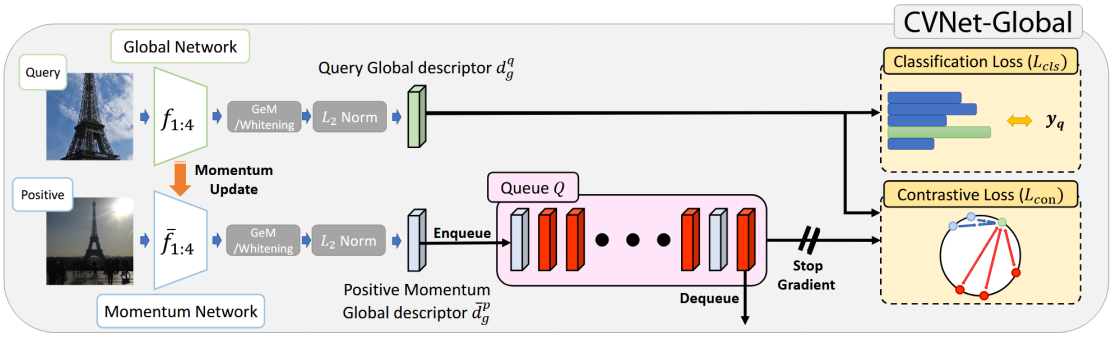


Figura 2.5: *CVNet-Global. Fuente: Lee et al. (2022)*

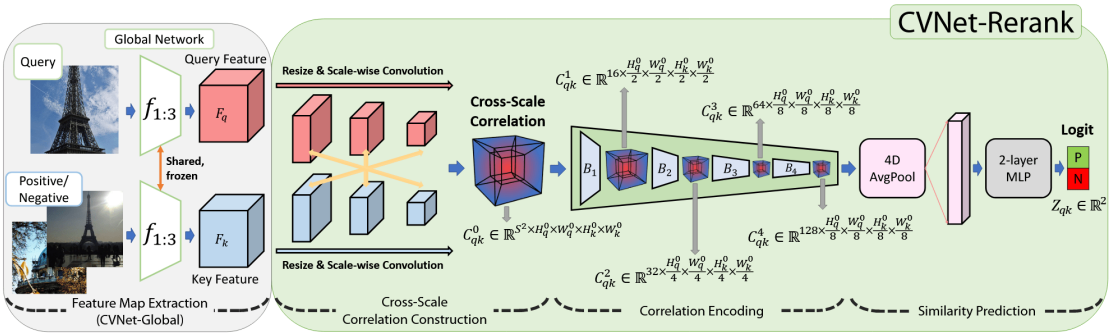


Figura 2.6: *CVNet-Rerank. Fuente: Lee et al. (2022)*

- *Scale*-GeM: extiende el *pooling* GeM para realizar inferencias multi-escala en un único paso.

Además, definieron un protocolo de *reranking* que solo emplea descriptores globales, demostrando que es posible refinar los resultados iniciales sin recurrir a características locales, lo cual simplifica y acelera el *pipeline* de reconocimiento de lugares. En la Fig. 2.7 puede verse el esquema completo.

2.3. Conjuntos de datos

Los puntos de interés turísticos presentan características particulares que hacen que los conjuntos de datos de imágenes generalistas —incluso de entornos urbanos— no resulten adecuados para entrenar los modelos requeridos en esta aplicación. En este contexto, Radenović et al. (2018a) publicaron *Revisiting Oxford and Paris*, una revisión de los benchmarks de 2007 y 2008, que contiene aproximadamente 5.000 imágenes de 11 sitios de Oxford y 6.000 de París. El protocolo de evaluación de $\mathcal{R}$Oxford y $\mathcal{R}$Paris tiene tres configuraciones posibles en función del tipo de imágenes incluidas:

- *Easy* (E): Las imágenes etiquetadas como *Easy* son consideradas positivas, mientras *Hard* y *Unclear* son ignoradas.

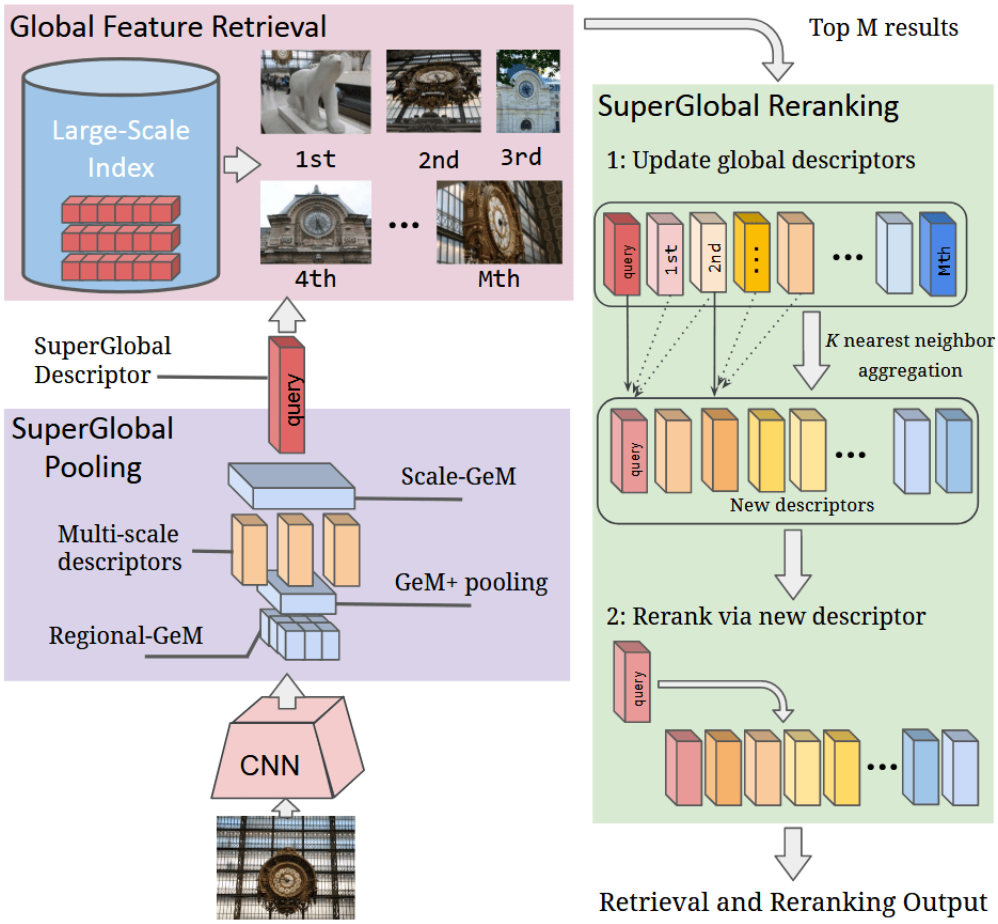


Figura 2.7: *SuperGlobal. Fuente: Shao et al. (2023)*

- *Medium* (M): Las imágenes etiquetadas como *Easy* y *Hard* son consideradas positivas, mientras las *Unclear* son ignoradas.
- *Hard* (H): Las imágenes etiquetadas como *Hard* son consideradas positivas, mientras *Easy* y *Unclear* son ignoradas.

La Fig. 2.8 muestra ejemplos de imágenes incluidas en el *dataset*. *Revisiting Oxford and Paris* se ha consolidado como referencia de evaluación. Debido a la cantidad limitada de imágenes y al sesgo proveniente de que todas las imágenes son de las mismas ciudades, estos datasets no son apropiados para el entrenamiento de redes profundas.

Para suplir esta carencia, Weyand et al. (2020) presentaron el *Google Landmarks Dataset v2* en las competiciones de Kaggle *Google Landmark Recognition 2019* y *Google Landmark Retrieval 2019*, con más de 5 millones de imágenes de 200.000 ubicaciones mundialmente distribuidas como se indica en la Fig. 2.9. No obstante, carece de anotaciones de *bounding boxes*, por lo que no permite entrenar directamente modelos de detección de objetos.

Por su parte, el dataset *Open Images V7* de Krasin et al. (2017) ofrece 1,9 millones de imágenes con 16 millones de *bounding boxes* en 601 clases de objetos, incluyendo categorías



Figura 2.8: *Muestra de imágenes de los datasets $\mathcal{R}$Oxford y $\mathcal{R}$Paris.* ***(a)*** *Imágenes de consulta.* ***(b)*** *Imágenes etiquetadas como Easy.* ***(c)*** *Imágenes etiquetadas como Hard.* ***(d)*** *Imágenes etiquetadas como Unclear. Fuente: Radenović et al. (2018a)*

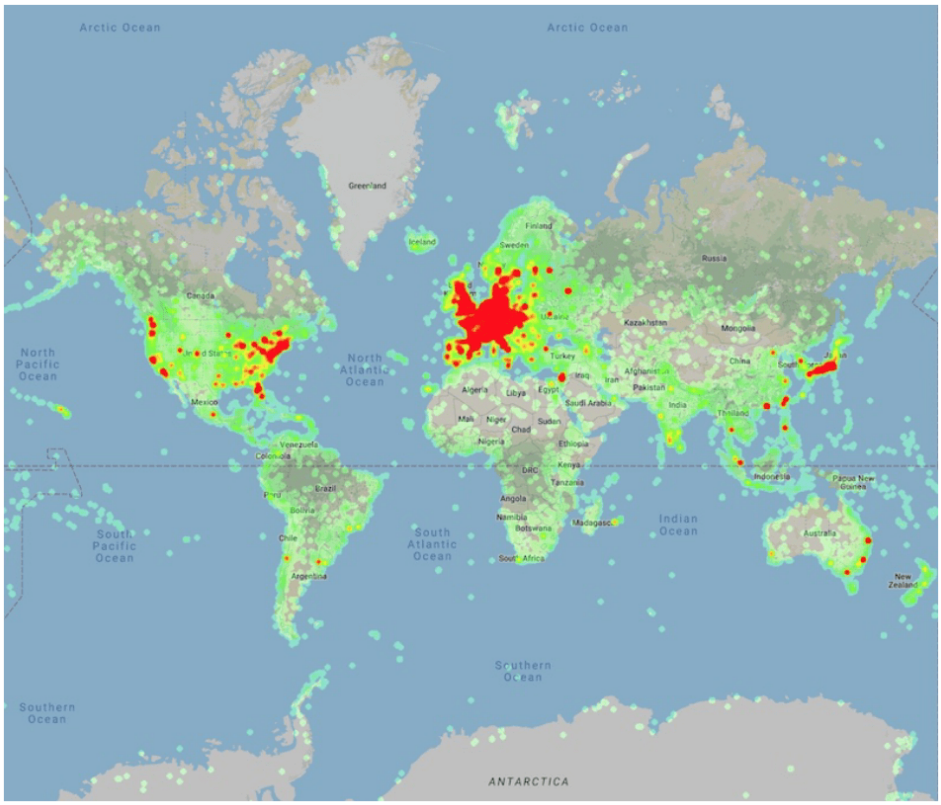


Figura 2.9: *Distribución geográfica del dataset Google Landmarks v2. Fuente: Weyand et al. (2020)*

relevantes para turismo como *bronze sculpture*, *building*, *bust*, *cannon*, *castle*, *fountain*, *house* o *lighthouse*.

Marco teórico

3

En el presente capítulo se detalla el *pipeline* completo de la solución con todas sus componentes y luego se profundiza en las bases teóricas de cada etapa. La Fig. 3.1 muestra el esquema completo de la aplicación. Primero se realiza una detección de objetos preservando únicamente aquellos que son relevantes como potenciales sitios de interés turístico. Luego se extraen las características de cada objeto para crear las representaciones compactas que permitirán buscar en la base de datos sitios similares y de esa forma identificar el *landmark* correspondiente.

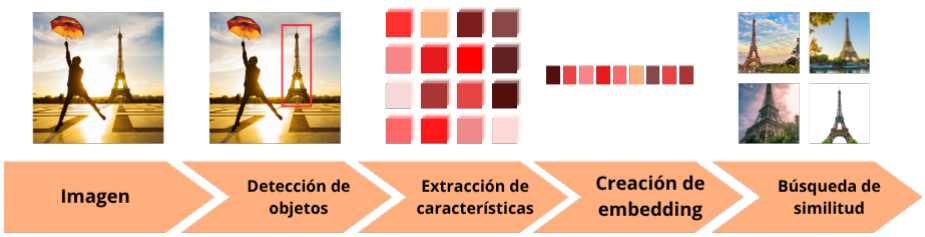


Figura 3.1: *Pipeline de detección de objetos con image retrieval. Elaboración propia.*

3.1. Detección de objetos

El objetivo del detector de objetos es identificar las regiones de la imagen que contienen un objeto perteneciente a una clase determinada. Estos detectores reciben una imagen como entrada y devuelven una lista de *bounding boxes*, cada una acompañada de un valor de confianza y las probabilidades asociadas a cada una de las clases con las que fue entrenado el modelo.

3.1.1. Bounding boxes

Las *bounding boxes* son cajas que encierran completamente al objeto detectado. Se representan mediante una lista de cuatro números que puede adoptar diferentes formatos como se muestra en la Fig. 3.2.

- **XYXY**: define la caja mediante un par de coordenadas (X_{min}, Y_{min}) y (X_{max}, Y_{max}), correspondientes a las esquinas superior izquierda e inferior derecha.
- **XYWH**: especifica las coordenadas (X, Y) de la esquina superior izquierda, junto con el ancho (W) y el alto (H) de la caja.
- **CXCYWH**: utiliza las coordenadas (C_X, C_Y) del centro de la caja, junto con su ancho (W) y alto (H).

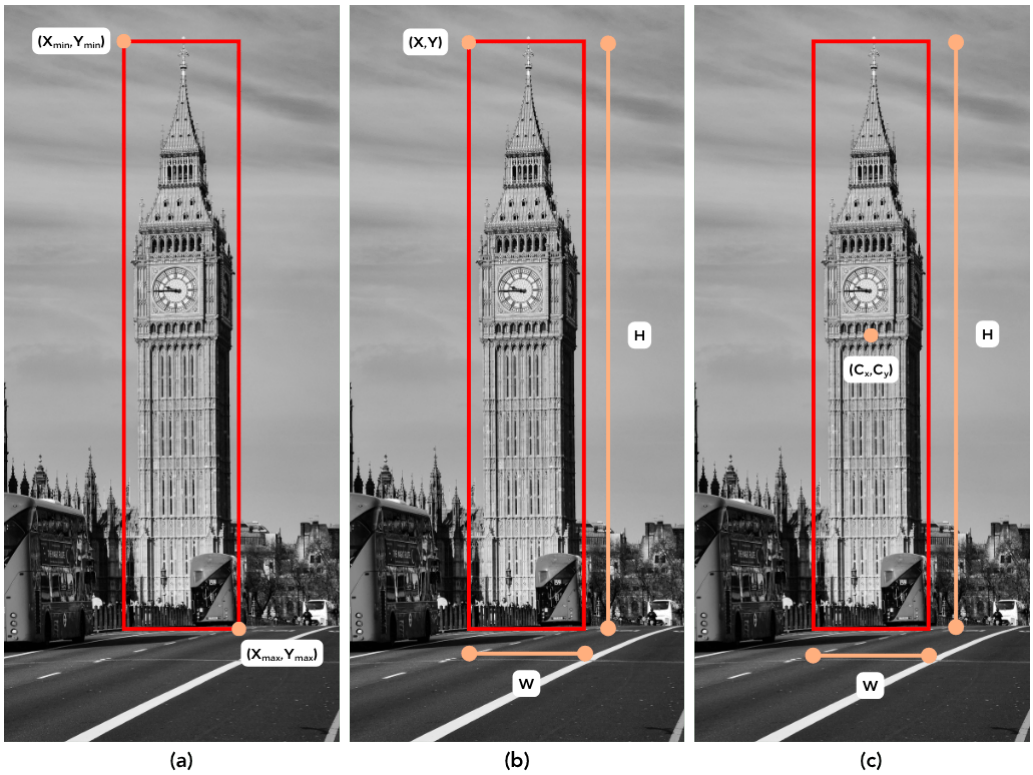


Figura 3.2: *Formatos para representación de bounding boxes.* ***(a) XYXY****,* ***(b) XYWH****,* ***(c) CXCYWH****. Elaboración propia.*

Es posible realizar la conversión entre estos formatos mediante transformaciones simples: De **XYWH** a **XYXY**:

$$X_{min} = X, \quad Y_{min} = Y, \quad X_{max} = X + W, \quad Y_{max} = Y + H$$

De **CXCYWH** a **XYXY**:

$$X_{min} = C_X - \frac{W}{2}, \quad X_{max} = C_X + \frac{W}{2}, \quad Y_{min} = C_Y - \frac{H}{2}, \quad Y_{max} = C_Y + \frac{H}{2}$$

3.1.2. Intersection over Union

Dadas dos *bounding boxes*, se puede utilizar la métrica *Intersection over Union* (IoU) para cuantificar el grado de solapamiento entre ellas. La IoU se define como el cociente entre el

área de intersección y el área de la unión de ambas cajas:

$$\text{IoU} = \frac{\text{Área}(\text{BOX}_1 \cap \text{BOX}_2)}{\text{Área}(\text{BOX}_1 \cup \text{BOX}_2)}$$

Un valor de IoU cercano a 1 indica un alto grado de coincidencia entre las dos cajas, mientras que valores cercanos a 0 indican poco o ningún solapamiento. Esta métrica se utiliza habitualmente para evaluar el rendimiento de los detectores de objetos comparando las predicciones con las anotaciones reales (*ground truth*). Además, IoU se emplea como criterio para eliminar detecciones redundantes o duplicadas en la salida de los detectores.

3.1.3. Non-Maximum Suppression

Los detectores de objetos generan muchas más *bounding boxes* de las que finalmente se utilizan. Por ejemplo, en el caso de YOLOv8 se generan 8400 cajas por inferencia. Muchas de estas cajas se encuentran fuertemente solapadas, representando la misma detección.

La técnica *Non-Maximum Suppression* (NMS) permite filtrar estas detecciones redundantes basándose en la confianza de las cajas y su grado de solapamiento medido con IoU. El procedimiento general del algoritmo es el que se muestra en el Algoritmo 1.

Algoritmo 1: Algoritmo de *Non-Maximum Suppression (NMS)*

1. Definir los umbrales: `score_threshold` e `IoU_threshold`.
2. Ordenar las *bounding boxes* en orden descendente por puntuación de confianza (*score*).
3. Eliminar todas las cajas con puntuación menor a `score_threshold`.
4. Para cada caja restante:
   a) Calcular la métrica IoU entre la caja actual y todas las demás.
   b) Eliminar aquellas cajas con IoU superior a `IoU_threshold`.
5. Devolver el conjunto final de *bounding boxes* seleccionadas.

De esta manera, se conservan únicamente las detecciones más representativas y se eliminan duplicados cercanos.

3.1.4. ROIPool y ROIAlign

Una vez obtenidas las *bounding boxes*, es necesario extraer para cada una de ellas un conjunto de características representado en tensores de tamaño fijo. Dado que las coordenadas de las *bounding boxes* están expresadas en punto flotante y referenciadas al tamaño original de la imagen, este paso no es trivial.

En el modelo *Fast* R-CNN de Girshick (2015) se propone el método *RoIPool*, que consiste en el Algoritmo 2.

Algoritmo 2: Algoritmo *RoIPool* de Girshick (2015)

1. Dado un *feature map* cuya dimensión de entrada fue reducida k veces y una salida deseada de tamaño $n \times n$.
2. Dividir las coordenadas de la *bounding box* (x_1, y_1, x_2, y_2) por k.
3. Redondear las coordenadas resultantes al entero más cercano.
4. Extraer del *feature map* el recorte correspondiente a esas coordenadas.
5. Dividir el recorte en una grilla de $n \times n$ celdas.
6. Aplicar *max pooling* sobre los valores en cada celda.

Si bien *RoIPool* es un método eficiente y simple, puede introducir errores de cuantización que afectan la precisión. Por ello, en *Mask* R-CNN, He et al. (2018) propone el método *RoIAlign* descripto en el Algortimo 3, que evita el redondeo y mejora la alineación espacial.

Algoritmo 3: Algoritmo *RoIAlign* de He et al. (2018)

1. Dado un *feature map* cuya dimensión de entrada fue reducida k veces y una salida deseada de tamaño $n \times n$.
2. Dividir las coordenadas de la *bounding box* (x_1, y_1, x_2, y_2) por k.
3. Dividir el recorte en una grilla de $n \times n$ celdas.
4. En cada celda, calcular el valor de $s \times s$ puntos mediante interpolación bilineal. Ver Fig. 3.3.
5. Obtener el valor de cada celda aplicando *max pooling* o *average pooling* sobre esos puntos.

Aunque *RoIAlign* es más preciso, también resulta más complejo y computacionalmente costoso. Por lo tanto, debe evaluarse si su uso se justifica para la aplicación concreta.

3.2. Extracción de características

Una vez detectadas las regiones relevantes en una imagen, el siguiente paso consiste en extraer características representativas que puedan describir su contenido visual de forma compacta y discriminativa. En visión por computadora, esto se logra comúnmente utilizando

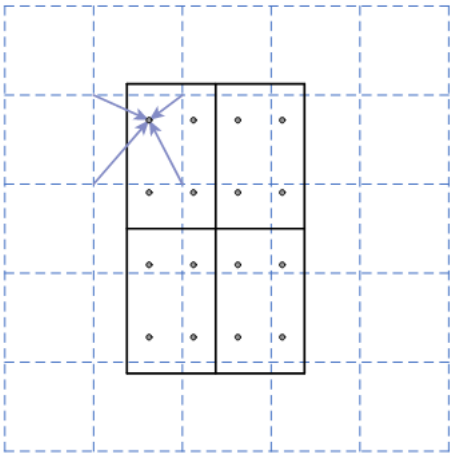


Figura 3.3: *ROIAlign: La grilla punteada representa el feature map, las líneas sólidas una bounding box dividida en bins (2×2 en este ejemplo), y los puntos los lugares de muestreo en cada celda. RoIAlign calcula el valor de cada punto de muestreo mediante interpolación bilineal sobre el feature map. Fuente: He et al. (2018)*

redes neuronales convolucionales (CNNs), que permiten capturar patrones jerárquicos en diferentes niveles de abstracción.

Estas redes actúan como *backbones* en arquitecturas modernas de reconocimiento visual y *retrieval*, y pueden ser seleccionadas según la tarea, el tamaño del modelo y la disponibilidad de pesos preentrenados. Algunos de los *backbones* más utilizados son VGG, ResNet, *EfficientNet*, y arquitecturas más recientes como ViT.

3.2.1. ResNet

ResNet50 es una arquitectura de red neuronal convolucional profunda introducida por He et al. (2015) como parte de la familia de *Residual Network* (ResNet). Su principal motivación fue resolver el problema del degradado de precisión observado al entrenar redes muy profundas, donde aumentar el número de capas no necesariamente mejora el rendimiento y puede incluso empeorarlo debido a la dificultad de propagar el gradiente durante el entrenamiento.

Para superar este problema, ResNet introduce bloques residuales, que incorporan conexiones directas (o *skip connections*) entre capas no adyacentes como se ve en la Fig. 3.4. Estas conexiones permiten que la red aprenda una función residual con respecto a la entrada, facilitando el aprendizaje de redes más profundas y estables. En lugar de aprender directamente una transformación $H(x)$, la red aprende una función residual $F(x) = H(x) - x$, de forma que la salida es $H(x) = F(x) + x$.

ResNet50 está compuesta por 50 capas de profundidad, organizadas en bloques de convolución y bloques residuales. Utiliza *kernels* pequeños (3x3 y 1x1), y aplica operaciones de *batch normalization* y ReLU tras cada convolución, seguidas de una suma con la entrada residual. Esta estructura modular permite una gran eficiencia y escalabilidad.

Gracias a su capacidad para aprender representaciones robustas y profundas, ResNet50

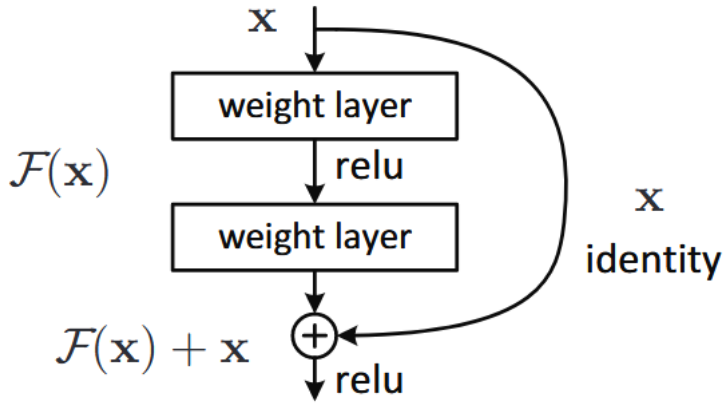


Figura 3.4: *Bloque genérico de ResNet. Fuente: He et al. (2015)*

se ha convertido en uno de los modelos estándar para tareas de clasificación y extracción de características, y es ampliamente utilizado como *backbone* en sistemas de reconocimiento y recuperación de imágenes.

3.2.2. Entrenamiento CVNet-Global

Dada una imagen de un potencial sitio de interés turístico $I \in \mathbb{R}^{3\times H\times W}$, se busca comprimir su contenido visual en un mapa de características $F \in \mathbb{R}^{C_t\times H_t\times W_t}$, a partir del cual se extrae un descriptor global $d_g \in \mathbb{R}^{C_g}$. Este descriptor global será utilizado para buscar en una base de datos otras imágenes similares que permitan identificar el sitio representado en la imagen de entrada.

Para que el descriptor d_g sea útil en tareas de reconocimiento de lugares, debe capturar únicamente las características invariantes de la escena, ignorando elementos variables como personas, vehículos u otros objetos móviles. Además, debe ser robusto frente a variaciones de iluminación, oclusiones y cambios de punto de vista.

Dos imágenes visualmente muy distintas pueden corresponder al mismo lugar, como las mostradas en la Fig. 3.5 (a). En ese caso, sus descriptores globales deberían ser lo más similares posible. Por el contrario, diferentes sitios de interés pueden compartir elementos visuales comunes, como ocurre en la Fig. 3.5 (b). En ese caso, es fundamental que sus descriptores sean lo más distintos posible para evitar confusiones.

Para alcanzar este objetivo, Lee et al. (2022) propusieron un método de entrenamiento llamado CVNet-Global. Este enfoque se basa en dos redes convolucionales idénticas: una actúa como *backbone* principal f, y la otra como red de *momentum* $\bar{f}$. Ambas comparten la misma arquitectura, basada en ResNet50, pero los parámetros de $\bar{f}$ se actualizan de forma progresiva a partir de los de f, siguiendo una estrategia de *momentum* para estabilizar el entrenamiento.

El entrenamiento utiliza dos funciones de pérdida complementarias diseñadas para maximizar la similitud entre imágenes del mismo lugar y minimizarla entre imágenes de lugares distintos. Estas pérdidas operan sobre los descriptores globales obtenidos por ambas redes



Figura 3.5: ***(a)*** *Imágenes visualmente diferentes ambas pertenecientes a la Biblioteca Bodleiana en Oxford.* ***(b)*** *Imágenes visualmente similares, arriba el Christ Church y abajo el edificio Balliol College ambos de la Universidad de Oxford. Fuente: Radenović et al. (2018a).*

y se combinan durante la retropropagación para optimizar el modelo principal f. El esquema propuesto se presenta en la Fig. 2.5.

Pérdidas por clasificación. Durante el entrenamiento, en cada iteración se utiliza una imagen de consulta I_q que se alimenta al *backbone* f para extraer el descriptor global d_g^q. A partir de este descriptor, se calcula un error de clasificación utilizando la función *CurricularFace-margined* propuesta en Huang et al. (2020), que modifica la similitud coseno mediante un margen adaptativo. La fórmula general es la siguiente:

$$\mathcal{L}_{cls} = -\log \frac{e^{C(W_{y_g}^T d_g^q, 1)/\tau}}{\sum_{i=1}^{N} e^{C(W_{y_i}^T d_g^q, \mathbb{1}_q^i)/\tau}}$$

donde:

W representa los pesos de clase,

τ es el parámetro de escala,

y_g es la clase verdadera,

$\mathbb{1}_{i=q}$ es un indicador que vale 1 si $y_i = y_g$ y 0 en caso contrario,

$C(\cdot)$ es la función que aplica el margen de *CurricularFace* sobre la similitud.

Pérdidas contrastiva. Además, se introduce una pérdida contrastiva basada en *momentum* para reforzar la discriminación entre lugares distintos. En cada iteración, se selecciona

una imagen positiva I_p con la misma etiqueta que I_q, la cual se procesa a través de la red de *momentum* f' para obtener un descriptor global positivo $d_g'^p$. Este descriptor se introduce en una cola Q que actúa como memoria dinámica y que contiene tanto descriptores positivos como negativos.

La pérdida contrastiva se calcula aplicando la función *CurricularFace* $\bar{C}$ sobre el conjunto de descriptores almacenados en la cola. La función $\bar{C}$ es idéntica a C, pero mantiene sus propios parámetros de promedio móvil. La fórmula general es la siguiente:

$$\mathcal{L}_{con} = \frac{-1}{|P(q)|} \sum_{p \in P(q)} \log \frac{e^{\bar{C}(d_g^q \cdot \bar{d}_g^p, 1)/\tau}}{\sum_{i \in \{p\} \cup N(q)} e^{\bar{C}(d_g^q \cdot \bar{d}_g^i, \mathbb{1}_q^i)/\tau}}$$

donde:

$\bar{C}$ es la versión *momentum* de la función *CurricularFace*,

$P(q)$ representa el conjunto positivo (al menos $d_g'^p$),

$N(q)$ representa el conjunto negativo en la cola,

Pérdidas totales. Esta combinación de pérdidas mejora la capacidad del modelo para generar descriptores globales robustos, discriminativos y consistentes a lo largo del entrenamiento.

Finalmente, se calcula el error total como la suma ponderada de ambas pérdidas:

$$\mathcal{L}_{total} = \lambda_{cls} \cdot \mathcal{L}_{cls} + \lambda_{con} \cdot \mathcal{L}_{con}$$

donde λ_{cls} y λ_{con} son coeficientes que controlan la contribución relativa de cada término.

La combinación de estos objetivos permite que la red aprenda tanto la variabilidad intra-clase como la distintividad entre clases, propiedades fundamentales para las tareas de recuperación de imágenes.

3.3. Creación del embedding

El método de *pooling* que se aplique sobre los mapas de características provenientes del *backbone* es fundamental para obtener un descriptor global robusto. Los métodos tradicionales de *pooling* son el *max pooling* y el *average pooling*, que consisten en extraer de cada mapa de características el valor máximo y promedio respectivamente.

3.3.1. GeM

Radenović et al. (2018b) propusieron un nuevo método de *pooling* denominado *Generalized-Mean pooling* (GeM). GeM está definido de la siguiente manera:

$$d_k = \left(\frac{1}{|\mathcal{X}_k|} \sum_{x \in \mathcal{X}_k} x^p \right)^{\frac{1}{p}}$$

donde $\mathcal{X}_k$ representa los valores del mapa de activación de la característica k y p es un parámetro que puede ser fijo o aprendible durante el entrenamiento. GeM permite interpolar entre diferentes estrategias de *pooling*:

- Si $p \to \infty$, GeM se comporta como *max pooling*.
- Si $p = 1$, GeM equivale a *average pooling*.

Esta flexibilidad hace que GeM sea particularmente útil en tareas de recuperación de imágenes, ya que permite adaptar el comportamiento del *pooling* a la naturaleza de los datos y al objetivo de la tarea.

3.3.2. SuperGlobal

Shao et al. (2023) propusieron *SuperGlobal*, un método para extraer el descriptor global de forma más robusta incorporando dos nuevos módulos que capturan características a nivel regional y multiescala. En la Fig. 2.7 se muestra el esquema propuesto para *SuperGlobal*.

El primer módulo es *Regional*-GeM, cuyo objetivo es capturar detalles locales que podrían perderse si se realiza un *pooling* directamente sobre todo el mapa de características. Mientras que GeM opera globalmente, *Regional*-GeM aplica un *pooling* tipo Lp de forma local sobre pequeñas regiones del *feature map*, mediante una operación de convolución sin parámetros entrenables. Esto genera un nuevo mapa de activación regional que se promedia con el mapa original. Posteriormente, se aplica GeM sobre esta combinación. Así se obtiene un descriptor global que refleja tanto las características dominantes como su disposición espacial en la imagen. La formulación matemática del proceso es la siguiente:

$$g_r = W \left(\frac{1}{H_d W_d} \sum_{h,w} \left(\frac{M_{h,w} + D_{h,w}}{2} \right)^p \right)^{\frac{1}{p}} + b$$

donde:

$g_r \in \mathbb{R}^{C_d}$ es el descriptor global resultante,

W y b hacen referencia a posibles pesos y sesgos de etapas posteriores del modelo,

D es el mapa de activación proveniente del *backbone*,

$M \in \mathbb{R}^{H_d \times W_d \times C_d}$ es el resultado de aplicar una convolución sobre D,

H_d y W_d son las dimensiones espaciales del mapa de activación,

p es un hiperparámetro que controla la intensidad del *pooling*.

Para obtener un descriptor global robusto frente a diferencias de escala, se aplican distintos escalados sobre la imagen de consulta. Para cada una de estas imágenes escaladas, se obtiene su descriptor global g_r. Luego, para combinar los descriptores correspondientes a las diferentes escalas se utiliza *Scale*-GeM.

Aplicar directamente GeM sobre estos descriptores globales puede fallar debido a la presencia de valores negativos en los vectores a combinar. Para resolver este problema, se

propone una versión modificada de GeM, diseñada específicamente para la inferencia multiescala. Su formulación es la siguiente:

$$g_{ms} = \left(\frac{1}{N} \sum_{s=1}^{N} (g_s + \zeta_s)^{p_{ms}} \right)^{1/p_{ms}} - \zeta_s$$

donde:

g_{ms} es el descriptor global resultante,

N es la cantidad de escalados realizados,

g_s son los descriptores obtenidos para cada imagen escalada,

$\zeta_s = -\min(g_s)$ es un desplazamiento aplicado a cada descriptor para evitar valores negativos,

p_{ms} es un hiperparámetro que regula la agregación.

Esta técnica permite integrar información multiescala en un solo descriptor global, incrementando la robustez del sistema ante cambios de tamaño y resolución en las imágenes de entrada.

3.4. Cálculo de similitud

Una vez obtenido el descriptor global de una imagen o de un objeto detectado, el siguiente paso es compararlo con los descriptores almacenados en la base de datos. Para ello, se utiliza comúnmente la similitud coseno, una medida que evalúa el ángulo entre dos vectores en el espacio.

La similitud coseno entre dos vectores x y y se define como:

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}$$

Este valor varía entre -1 y 1, donde 1 indica que los vectores apuntan en la misma dirección (máxima similitud), 0 que son ortogonales (sin similitud), y -1 que apuntan en direcciones opuestas.

Una ventaja importante es que si todos los descriptores (tanto las *queries* como los de la base de datos) son previamente normalizados con norma L2, entonces la fórmula de similitud coseno se simplifica al producto vectorial:

$$\text{sim}(x, y) = x^T y$$

Esta simplificación permite calcular la similitud de múltiples objetos detectados en una imagen con todos los descriptores de la base de datos en una sola operación matricial eficiente:

$$\text{sim}(Q, X) = Q \cdot X^T$$

donde Q es una matriz que contiene los descriptores de los objetos detectados (una fila por cada uno), y X es la matriz que contiene los descriptores de la base de datos.

Este enfoque no solo es computacionalmente eficiente, sino que también facilita la búsqueda de los k vecinos más cercanos de cada objeto detectado en tiempo real.

3.5. Aplicación Android con realidad aumentada

El desarrollo de soluciones basadas en visión por computadora en entornos móviles requiere considerar aspectos específicos del sistema operativo, las capacidades de ejecución de modelos y las tecnologías de interacción con el entorno físico. En esta sección se introducen los conceptos fundamentales asociados al desarrollo en Android, al uso del formato ONNX para la inferencia de modelos de aprendizaje profundo y a la integración de elementos de realidad aumentada.

3.5.1. Android Studio

Android Studio es el entorno oficial de desarrollo integrado (IDE) proporcionado por Google para la creación de aplicaciones sobre la plataforma Android. Está basado en IntelliJ IDEA e incluye un conjunto de herramientas diseñadas específicamente para facilitar la creación, depuración y optimización de aplicaciones móviles. Entre sus características principales se encuentran un emulador de dispositivos, herramientas de análisis de rendimiento, asistentes de diseño de interfaces gráficas y soporte para los lenguajes Kotlin y Java. Su integración con el sistema de construcción Gradle permite gestionar dependencias y automatizar tareas de compilación, facilitando el desarrollo modular de aplicaciones complejas.

3.5.2. Open Neural Network Exchange

Open Neural Network Exchange (ONNX) es un formato abierto diseñado para representar modelos de aprendizaje profundo de manera interoperable entre distintos *frameworks*. Fue creado por Microsoft y Facebook con el objetivo de facilitar el traslado de modelos entre herramientas como PyTorch, TensorFlow, MXNet y otros. Un modelo exportado a ONNX puede ser ejecutado en diversos dispositivos utilizando ONNX *Runtime*, un motor de inferencia optimizado que soporta aceleración por hardware y entornos con recursos limitados, como los dispositivos móviles. ONNX permite así desplegar modelos previamente entrenados en entornos de producción sin necesidad de redefinir su arquitectura o comportamiento.

3.5.3. Realidad aumentada

La Realidad Aumentada (RA) es una tecnología que permite superponer información digital al entorno físico percibido a través de un dispositivo, generalmente una cámara en tiempo real. A diferencia de la realidad virtual, que genera un entorno completamente simulado, la RA complementa la experiencia del mundo real añadiendo elementos visuales, sonoros o interactivos que se integran espacialmente con la escena captada por el dispositivo. En el

contexto de este trabajo, la RA se utiliza para mostrar sobre la imagen de la cámara los puntos de interés turístico detectados, generando una experiencia enriquecida y contextualizada para el usuario. *Frameworks* como ARCore permiten implementar estas capacidades en dispositivos Android, facilitando la estimación de superficies, la detección de planos y el anclaje de objetos virtuales en el espacio físico.

Materiales y métodos

4

En este capítulo se detallan los recursos empleados y las etapas seguidas para el desarrollo del sistema propuesto. En primer lugar, se describen los materiales utilizados, tanto a nivel de *software* como de *hardware*. A continuación, se expone la metodología adoptada para implementar la solución, estructurada según el modelo CRISP-DM. Esta metodología permite abordar de forma sistemática todas las fases del desarrollo, desde la comprensión del problema y la preparación de los datos, pasando por el modelado y la evaluación del sistema, hasta llegar al despliegue final del modelo en dispositivos móviles.

4.1. Materiales

En esta sección se detallan los recursos de *software* y *hardware* utilizados para el desarrollo de este proyecto.

4.1.1. Software

El desarrollo del *pipeline* se realizó utilizando el lenguaje de programación Python 3.11 en un entorno de trabajo basado en Anaconda. La implementación del código se llevó a cabo en Visual Studio Code, mientras que las pruebas experimentales y visualizaciones interactivas se efectuaron en Jupyter Notebook. Para el desarrollo de la aplicación móvil se empleó Android Studio, utilizando el lenguaje Kotlin. La integración del modelo dentro de la aplicación se realizó mediante ONNX *Runtime*, utilizando el conjunto de operaciones definido en el opset 16.

El *pipeline* se construyó empleando la librería PyTorch. Para la detección de objetos se utilizó la versión oficial de la librería de Ultralytics, implementando un modelo YOLOv8 preentrenado con el dataset *Open Images V7*, seleccionado por su riqueza de clases y relevancia para el dominio turístico. Como extractor de características visuales se utilizó un *backbone* derivado del repositorio oficial *Correlation Verification for Image Retrieval* propuesto en Lee et al. (2022), el cual fue adaptado y cargado con pesos preentrenados sobre el *Google Landmarks Dataset v2*. Asimismo, se incorporó una variante modificada del método de *pooling* presentado en *Global Features are All You Need for Image Retrieval and Reranking* de Shao et al. (2023) para la obtención de descriptores robustos.

La evaluación del rendimiento del sistema se realizó utilizando el *benchmark Revisited*

Oxford and Paris, siguiendo su protocolo oficial.

4.1.2. Hardware

Para el desarrollo y evaluación del pipeline se utilizó un equipo con procesador Intel(R) Core(TM) i5-9300HF CPU @ 2.40GHz, 8 GB de memoria RAM, sistema operativo Windows 11 y una tarjeta gráfica NVIDIA GeForce GTX 1650. El despliegue y prueba de la aplicación móvil se efectuó en un dispositivo Google Pixel 8 Pro, utilizado como entorno de ejecución para validar el funcionamiento del sistema en escenarios reales.

4.2. Metodología

Para guiar de forma estructurada y replicable el desarrollo de este trabajo, se adopta el modelo de proceso *Cross-Industry Standard Process for Data Mining* (CRISP-DM) propuesto por Chapman et al. (2000). Este modelo proporciona un marco ampliamente aceptado para proyectos de minería de datos y aprendizaje automático, abarcando todo el ciclo de vida desde la comprensión del problema hasta la implementación de la solución.

CRISP-DM se estructura en seis fases interconectadas. Aunque existe una secuencia típica, el proceso no es lineal ni rígido: es común iterar entre fases según los resultados obtenidos. En la Fig. 4.1 se muestra un esquema general del proceso y sus relaciones cíclicas.

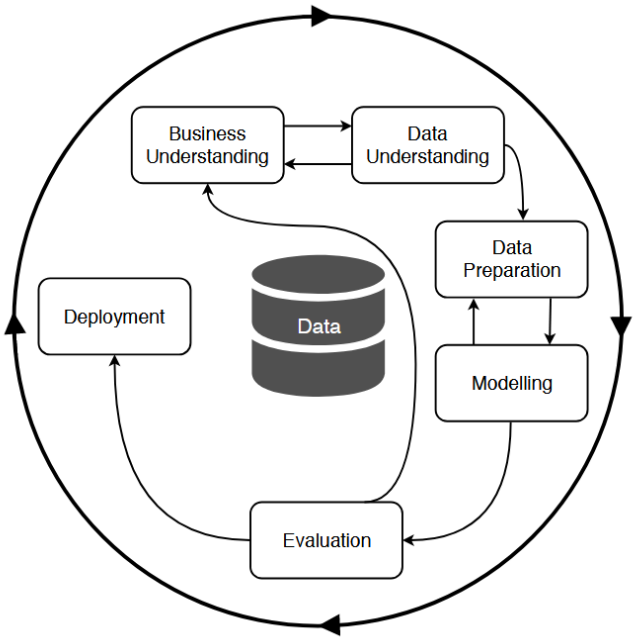


Figura 4.1: *Esquema de la metodología CRISP-DM. Fuente: Martinez-Plumed et al. (2021)*

4.2.1. Comprensión del negocio

El objetivo general de este proyecto es implementar un sistema móvil de visión por computadora capaz de detectar puntos de interés turístico y operar en entornos de realidad aumentada, permitiendo la incorporación de nuevos lugares sin requerir reentrenamiento del modelo. Este enfoque se materializa en una solución que integra técnicas de *object detection*, *place recognition* e *instance image retrieval*, desarrollada con criterios de eficiencia, escalabilidad y portabilidad.

Desde la perspectiva del uso real, se contempla que los turistas suelen disponer de acceso limitado a datos móviles y, en muchas ocasiones, enfrentan condiciones adversas para el uso de GPS, como ocurre en interiores, zonas con señal débil o entornos urbanos densamente construidos. Por tanto, el sistema debe ser completamente funcional de manera local, sin depender de conectividad ni geolocalización. El usuario interactuará con la aplicación enfocando con la cámara una escena del entorno, en la cual el sistema debe ser capaz de detectar, identificar y mostrar en tiempo real los puntos de interés presentes mediante una superposición visual a través de realidad aumentada. Esta interacción debe ser ágil, precisa y resiliente a variaciones en iluminación, ángulo u oclusión parcial de los elementos turísticos.

Además, se considera un requisito fundamental que la incorporación de nuevos *landmarks* al sistema sea sencilla y accesible para operadores sin conocimientos técnicos avanzados. Para ello, el diseño debe permitir registrar nuevos puntos de interés a partir de un conjunto reducido de imágenes, en el orden de 10 muestras, manteniendo un buen desempeño sin necesidad de reentrenar el modelo base.

4.2.2. Comprensión de los datos

Para el desarrollo del sistema propuesto se hizo uso de diferentes fuentes de datos, cada una seleccionada en función de su utilidad en una etapa específica del pipeline. En total se utilizaron tres datasets de dominio público y uno personalizado, cubriendo tareas de detección, extracción de características y evaluación.

4.2.2.1. Datos para detección de objetos

En primer lugar, se utilizó el conjunto de datos *Open Images V7* para la tarea de detección de objetos. Este *dataset* proporciona una gran variedad de imágenes anotadas con cajas delimitadoras y etiquetas de clase, tal como se detalla en la Tabla 4.1. Muchas de estas clases resultan especialmente relevantes en el contexto turístico, al incluir elementos como monumentos, edificios históricos y estructuras urbanas. En la Fig. 4.2 se presentan algunas imágenes de ejemplo.

El subconjunto utilizado para detección contiene aproximadamente 1,9 millones de imágenes, con un total de 16 millones de *bounding boxes* distribuidas en 601 clases diferentes. El *dataset* puede ser descargado desde esta URL: https://storage.googleapis.com/openimages/web/download_v7.html.



Figura 4.2: *Muestra de imágenes turísticas de Open Images V7. Fuente: Krasin et al. (2017)*

Tabla 4.1: *Anotaciones en Open Images V7. Fuente: Krasin et al. (2017)*

Campo	Definición
ImageID	Identificador de la imagen a la que corresponde la *bounding box*.
LabelName	Clase a la que corresponde el objeto.
XMin	Extremo izquierdo de la *bounding box* en el rango [0, 1].
XMax	Extremo derecho de la *bounding box* en el rango [0, 1].
YMin	Extremo superior de la *bounding box* en el rango [0, 1].
YMax	Extremo inferior de la *bounding box* en el rango [0, 1].

4.2.2.2. Datos para extracción de características

Para la etapa de extracción de características, se empleó el *Google Landmarks Dataset v2*, un conjunto masivo y diverso que contiene más de 5 millones de imágenes correspondientes a más de 200 mil puntos de interés turístico de todo el mundo. Su distribución geográfica amplia y su enfoque en lugares reales lo convierten en una base ideal para modelos capaces de generar descriptores robustos e invariantes. En la Fig. 4.3 se muestran ejemplos de imágenes pertenecientes al conjunto de datos. La Tabla 4.2 muestra la estructura de las etiquetas y los metadatos del *dataset*. El *dataset* puede ser descargado desde esta URL: `https://storage.googleapis.com/gld-v2/web/index.html`.

4.2.2.3. Datos para evaluación

La evaluación del sistema se llevó a cabo utilizando el *benchmark Revisited Oxford and Paris*, el cual proporciona un protocolo estandarizado para la comparación de algoritmos de *image retrieval* aplicados al reconocimiento de lugares. Este *benchmark* es ampliamente utilizado en la literatura y permite medir el rendimiento del sistema en distintos niveles de



Figura 4.3: *Muestra de imágenes de Google Landmarks Dataset v2. Fuente: Weyand et al. (2020)*

Tabla 4.2: *Anotaciones en Google Landmarks Dataset v2. Fuente: Weyand et al. (2020)*

Archivo: `train.csv`	
Campo	**Definición**
`id`	Identificador de la imagen.
`url`	*Link* a la imagen.
`landmark_id`	Identificador del *landmark*.
Archivo: `train_label_to_category.csv`	
Campo	**Definición**
`landmark_id`	Identificador del *landmark*.
`category`	*Link* a la página de Wikimedia con la definición de la clase.

dificultad mediante métricas bien definidas.

Revisited Oxford and Paris está compuesto por dos conjuntos de datos distintos. Por un lado, el *Oxford Buildings Dataset* contiene 5.062 imágenes de la ciudad de Oxford, correspondientes a 11 *landmarks* diferentes. Por otro, el *Paris Buildings Dataset* incluye 6.412 imágenes de la ciudad de París, también organizadas en 11 *landmarks* distintos.

Como se describió en la Sección 2.3, las imágenes están etiquetadas según el nivel de dificultad que presentan para su reconocimiento. La Tabla 4.3 muestra cómo están estructuradas las anotaciones del conjunto de datos. Las imágenes, anotaciones y el código fuente pueden descargarse desde el repositorio oficial del benchmark: `https://github.com/filipradenovic/revisitop`.

Tabla 4.3: *Anotaciones en Revisited Oxford and Paris. Fuente: Radenović et al. (2018a)*

Archivos: `gnd_roxford5k.pkl` y `gnd_rparis6k.pkl`	
Campo	**Definición**
`imlist`	Lista de imágenes de la base de datos.
`qimlist`	Lista de imágenes de consulta.
`gnd`	Lista de diccionarios con el *ground truth*.
Estructura del *ground truth*:	
Campo	**Definición**
`bbx`	*Bounding box de la imagen de consulta*.
`easy`	Lista de índices de imágenes de `imlist` catalogadas como *Easy*.
`hard`	Lista de índices de imágenes de `imlist` catalogadas como *Hard*.
`junk`	Lista de índices de imágenes de `imlist` catalogadas como *Junk*.

4.2.2.4. Datos para prueba de campo

Finalmente, se construyó un dataset personalizado con imágenes de la ciudad de Cáceres (España), con el fin de validar el funcionamiento del sistema en un entorno real. Este conjunto se utilizó para pruebas de campo y verificación de la capacidad del modelo para operar de manera efectiva sobre escenas reales captadas con un dispositivo móvil.

4.2.3. Preparación de los datos

Se desarrolló un pipeline modular para procesar imágenes y generar una base de datos estructurada de descriptores visuales. Esta fase incluyó la carga y normalización de imágenes, la identificación de regiones relevantes dentro de cada imagen a través de anotaciones de detección, el recorte y redimensionamiento de dichas regiones a resoluciones estándar, y la organización del resultado en dos estructuras paralelas: un *DataFrame* con metadatos (nombre de imagen, clase, confianza, coordenadas de la región) y una matriz con los vectores de características extraídos. Este proceso fue diseñado para ser reutilizable, escalable y compatible con futuras integraciones en entornos de inferencia.

4.2.4. Modelado

El objetivo final de esta etapa es obtener un grafo en formato ONNX que reciba como entrada una imagen y una base de datos de descriptores, y que devuelva como salida una lista de *bounding boxes* con la identificación del lugar correspondiente a cada una de ellas.

Para lograrlo, se construyeron cinco grafos independientes, cada uno correspondiente a una etapa del proceso: preprocesado, detección, extracción, búsqueda y postprocesado. Cada grafo fue diseñado de forma que los nodos de salida de una etapa se conecten directamente con las entradas del grafo siguiente, permitiendo una ejecución secuencial sin

intervención externa. En los casos en que cierta información es requerida por etapas posteriores, se implementaron mecanismos de *bypass* para transportar esos datos a través de los subgrafos intermedios. La Fig. 4.4 muestra el esquema completo del *pipeline* implementado.

Cada subgrafo fue implementado en PyTorch como una clase que extiende la clase base `Module`. En la fase de inicialización (`__init__`) se definen todos los parámetros que configuran el comportamiento del modelo en esa etapa. Una vez exportada la clase a formato ONNX, el grafo resultante queda estructurado de acuerdo con dicha configuración, y los parámetros utilizados dejan de ser modificables. Además, cada clase implementa un método `forward`, responsable de ejecutar las operaciones específicas asociadas a la etapa correspondiente dentro del *pipeline* (ver Apéndice A).

En los siguientes apartados se detalla la implementación de cada uno de los subgrafos.

4.2.4.1. Preprocesado

El módulo de preprocesado cuenta con un único parámetro de configuración, denominado `image_dim`, que define el tamaño al que deben ser redimensionadas las imágenes de entrada.

Su método `forward` realiza las siguientes operaciones:

- Extrae las dimensiones originales de la imagen de entrada, almacenándolas en la variable `orig_size`.
- Permuta los canales de color de BGR a RGB.
- Redimensiona la imagen a `image_dim` mediante interpolación bilineal.
- Normaliza los valores de píxeles al rango $[0, 1]$.

Como salida, el módulo devuelve la imagen procesada y el valor de `orig_size`. Este último será necesario en la etapa de postprocesado para reescalar las *bounding boxes* al tamaño original de la imagen. Por este motivo, todos los subgrafos intermedios incorporan un mecanismo de *bypass* que transporta este valor a lo largo del grafo completo.

4.2.4.2. Detección

Para la detección de objetos se utilizó el modelo YOLO, seleccionado por su alta velocidad de inferencia, lo que lo hace especialmente adecuado para aplicaciones en tiempo real. En particular, se empleó la versión YOLOv8n, que ofrece un buen equilibrio entre precisión y eficiencia computacional. Esta versión cuenta, además, con pesos preentrenados sobre el dataset *Open Images V7*, disponibles directamente a través de la plataforma de Ultralytics.

El módulo YOLO proporcionado por Ultralytics incluye una funcionalidad integrada para exportar el modelo al formato ONNX. El grafo resultante requiere únicamente como entrada la imagen ya preprocesada y devuelve como salida un tensor denominado `detections`. Este tensor tiene forma $(1, 4 + C, N)$, donde:

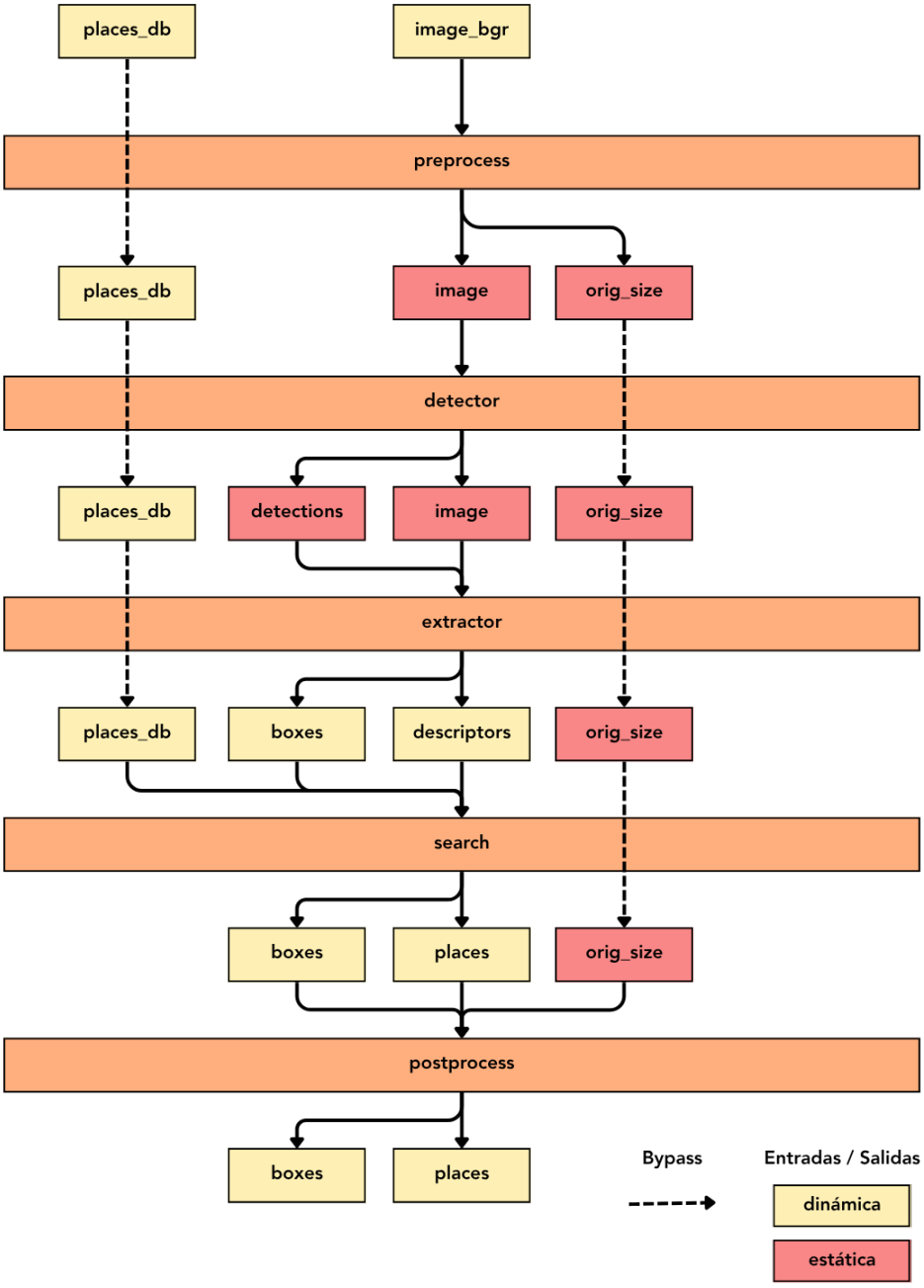


Figura 4.4: *Esquema del pipeline completo. Elaboración propia*

- La primera dimensión corresponde al *batch size*, que en esta aplicación es siempre igual a uno.
- La segunda dimensión representa los 4 valores que definen la *bounding box* en formato **CXCYWH** (centro x, centro y, ancho, alto), seguidos por los *scores* de clasificación para cada una de las clases. En este caso, el número de clases C es 601.

- La tercera dimensión N indica la cantidad total de predicciones generadas por el modelo. Al exportar el modelo a ONNX, no se realiza ningún filtrado por umbral de *score* ni se aplica NMS. En YOLOv8, este valor es fijo y corresponde a 8.400.

4.2.4.3. Extracción

El módulo de extracción recibe como entrada el tensor `detections` junto con la imagen preprocesada, y se encarga de generar descriptores visuales robustos para cada región detectada. Este módulo se compone de tres etapas principales: procesado de detecciones, extracción de características y *pooling*. Cada una de estas etapas cuenta con sus propios parámetros de configuración.

Procesado de detecciones. Esta etapa incluye los siguientes parámetros: `allowed_classes`, `score_thresh`, `iou_thresh`, `scales`, `mean` y `std`. En primer lugar, se transforman las *bounding boxes* desde el formato **CXCYWH** (centro x, centro y, ancho, alto) al formato **XYXY** (esquina superior izquierda y esquina inferior derecha). A continuación, se aplica NMS utilizando el umbral definido por `iou_thresh` para eliminar cajas superpuestas. Luego, se descartan todas las detecciones cuyo *score* sea inferior a `score_thresh`, así como aquellas cuya clase no se encuentre en la lista de `allowed_classes`.

Sobre las cajas válidas restantes —a las que se suma una caja adicional que cubre toda la imagen— se generan múltiples escalas según los factores definidos en `scales`. Este paso permite aumentar la robustez del sistema frente a variaciones de tamaño y distancia entre el usuario y el objeto. A partir de estas cajas escaladas, se recortan las regiones correspondientes de la imagen original mediante *RoIPool*, y se redimensionan a un tamaño fijo de 224×224 píxeles. Finalmente, las imágenes recortadas se normalizan utilizando los valores de `mean` y `std`. El resultado de este proceso es un tensor de forma $(M \times K, 3, 224, 224)$, donde M representa el número de cajas filtradas (incluyendo la imagen completa) y K la cantidad de escalas por caja.

Extracción de características. La extracción de características se realiza utilizando una red ResNet50 preentrenada con el método CVNet-Global, descrito en la Sección 3.2.2. Tanto los pesos como la arquitectura de la red fueron descargados del repositorio oficial del artículo *Correlation Verification for Image Retrieval* de Lee et al. (2022), disponible en `https://github.com/sungonce/CVNet/`. La implementación fue ligeramente modificada para adaptarla a los requerimientos del presente trabajo, en particular mediante la sustitución del método de *pooling*. Al aplicar la ResNet50 sobre el tensor de entrada, se obtiene un mapa de activaciones de tamaño $(M \times K, 2048, 7, 7)$.

Pooling. La última etapa del módulo es la agregación de características mediante el método *SuperGlobal*, detallado en la Sección 3.3.2. Esta técnica combina tres tipos de *pooling* sucesivos y cuenta con seis parámetros configurables: `rgem_pr`, `rgem_size`, `gem_p`, `sgem_ps`, `sgem_infinity` y `eps`. La implementación se basó en el código del artículo *Global Features*

are All You Need for Image Retrieval and Reranking de Shao et al. (2023), adaptado para garantizar compatibilidad con la exportación a ONNX.

El procedimiento se desarrolla en tres fases:

1. **Regional-GeM:** se divide el mapa de características en regiones definidas por una grilla de tamaño `rgem_size` y se aplica un GeM regional con el parámetro `rgem_pr`. El resultado se combina con el mapa original mediante promediado.

2. **GeM global:** sobre el resultado anterior se aplica un nuevo GeM, esta vez global, con el parámetro `gem_p`, obteniendo un tensor de dimensión $(M \times K, 2048)$.

3. **Scale-GeM:** finalmente, se agregan los descriptores correspondientes a cada imagen a través de las escalas definidas. Si el parámetro `sgem_infinity` está activado, se aplica un *max pooling*; en caso contrario, se aplica una agregación basada en potencia con el parámetro `sgem_ps`, siguiendo el procedimiento descrito en la Sección 3.3.2.

El resultado final de esta etapa es un tensor de forma $(M, 2048)$ normalizado L2, que contiene los descriptores visuales globales correspondientes a cada región detectada.

4.2.4.4. Búsqueda

Una vez obtenidos los descriptores visuales de cada objeto detectado en la imagen, se procede a realizar una búsqueda en la base de datos para determinar si dichas detecciones corresponden con lugares conocidos. El módulo de búsqueda requiere tres entradas: `boxes`, `descriptors` y `places_db`. Los dos primeros se corresponden con las salidas del módulo de extracción, mientras que `places_db` es un tensor precomputado que contiene los descriptores de referencia de la base de datos.

Este tensor `place_db` se genera de forma *offline* ejecutando el mismo pipeline de detección y extracción sobre cada imagen de la base de datos, hasta la salida del extractor. Durante este proceso se aplican dos filtros adicionales: se descartan todas las cajas cuya área sea inferior a un umbral `min_area`, y se eliminan aquellos descriptores que presentan baja similitud con respecto al resto de los descriptores de la misma imagen, controlado por el parámetro `min_sim`. Estos filtros buscan asegurar que los descriptores almacenados sean representativos y distintivos de los puntos de interés reales. En el Apéndice B se describe la forma de crear la base de datos.

El módulo de búsqueda cuenta con los siguientes parámetros de configuración: `topk`, `min_sim`, `min_votes`, `remove_inner_boxes` y `join_boxes`. El primer paso consiste en calcular la similitud entre los descriptores de consulta y los descriptores de la base de datos mediante producto escalar, que equivale a la similitud coseno si los vectores están normalizados, como se explicó en la Sección 3.4. A partir de esta matriz de similitud, se seleccionan para cada detección los `topk` resultados más similares, descartando aquellos cuya puntuación sea inferior a `min_sim`.

Sobre las coincidencias válidas obtenidas para cada detección, se realiza una votación por mayoría para determinar a qué *landmark* corresponde. Si el porcentaje de votos a favor

de un mismo *landmark* es inferior al umbral definido por `min_votes`, se considera que no hay información suficiente para establecer una correspondencia confiable con la base de datos, y la detección se descarta.

En el paso final, se realiza una depuración de las cajas generadas. Si el parámetro `remove_inner_boxes` toma un valor dentro del rango $[0, 1]$, se eliminan aquellas cajas más grandes que se solapen significativamente con cajas más pequeñas (según el umbral indicado), conservando así las detecciones más específicas. Además, si el parámetro `join_boxes` está activado, todas las cajas que hayan sido asociadas al mismo lugar se agrupan en una única *bounding box* que las engloba.

La salida del módulo consiste en el conjunto final de *bounding boxes*, junto con la identificación del lugar asociado a cada una de ellas.

4.2.4.5. Postprocesado

Al inicio del *pipeline*, la imagen de entrada fue redimensionada al tamaño definido por el parámetro `image_dim`. Como consecuencia, todas las bounding boxes generadas en las etapas posteriores están referenciadas a estas dimensiones escaladas.

El objetivo del módulo de postprocesado es transformar dichas cajas de vuelta al sistema de coordenadas original de la imagen. Para ello, se utiliza el valor `orig_size`, capturado durante el preprocesado, que contiene las dimensiones reales de la imagen original. A partir de esta información, se calcula un vector de escala que permite ajustar proporcionalmente las coordenadas de cada bounding box y restaurarlas a su posición correcta sobre la imagen de entrada.

4.2.5. Evaluación

Se evaluó el rendimiento del sistema sobre los *benchmarks* estándar *Revisited Oxford and Paris*, utilizando métricas como **mAP** (*Mean Average Precision*) y **Precision@k**, y diferenciando niveles de dificultad (*easy*, *medium*, *hard*). Este *benchmark* está diseñado para sistemas de *image retrieval* sin detección de objetos. Por ese motivo las imágenes de los *datasets* contienen solo una referencia de lugar. Adicionalmente se han agregado métricas de tiempos de inferencia ya que resulta crítico para el funcionamiento de la aplicación en tiempo real.

Para evaluar la *performance* del sistema en un escenario con varios *landmarks* en una misma imagen se contruyó un *dataset* con imágenes de la ciudad de Cáceres. Este dataset también permitió probar la aplicación en un entorno real utilizando la cámara de un teléfono móvil.

4.2.6. Despliegue

Como resultado del desarrollo, se obtuvo un sistema completamente exportado al formato ONNX, que integra en un único grafo de inferencia todas las etapas del *pipeline*: preprocesado, detección, extracción, búsqueda y postprocesado. Este modelo puede ejecutarse en

entornos móviles o de realidad aumentada sin requerir reentrenamiento al incorporar nuevas imágenes, cumpliendo así con el objetivo de adaptabilidad planteado.

Para validar su funcionamiento, se desarrolló una aplicación móvil utilizando Android Studio. El modelo ONNX fue integrado en la aplicación mediante ONNX *Runtime*, lo que permite ejecutar las inferencias directamente en el dispositivo. La aplicación recibe las imágenes captadas en tiempo real por la cámara del teléfono y las envía al modelo, junto con la base de descriptores de lugares conocidos. Como resultado, se obtienen las coordenadas y las clases asociadas a los objetos detectados. A partir de esta información, la aplicación identifica los nombres de los lugares correspondientes y los superpone sobre la imagen capturada, permitiendo una visualización en tiempo real mediante realidad aumentada.

Resultados

5

En este capítulo se presentan los resultados obtenidos al aplicar el modelo propuesto, descrito previamente en la Sección 4, con el objetivo de evaluar su desempeño en tareas de reconocimiento de lugares.

5.1. Parámetros del sistema

La Tabla 5.1 resume los parámetros configurables del sistema, organizados según la etapa del *pipeline* en la que intervienen.

Tabla 5.1: *Lista de parámetros del modelo.*

Parámetro	Etapa	Tipo	Descripción
`image_dim`	Preprocesado de detección	`tuple[int]`	Dimensiones a las que se redimensiona la imagen de entrada.
`allowed_classes`	Postprocesado de detección	`list[int]`	Índices de clases permitidas para conservar detecciones.
`score_thresh`	Postprocesado de detección	`float`	Umbral mínimo de confianza para conservar una detección.
`iou_thresh`	Postprocesado de detección	`float`	Umbral de solapamiento para aplicar NMS.
`scales`	Preprocesado de extracción	`list[float]`	Escalas aplicadas a cada detección para generar variantes.
`mean`	Preprocesado de extracción	`list[float]`	Media para normalización de imágenes (por canal RGB).
`std`	Preprocesado de extracción	`list[float]`	Desviación estándar para normalización (por canal RGB).
`rgem_pr`	*Pooling*	`float`	Potencia utilizada en *Regional*-GeM.
`rgem_size`	*Pooling*	`int`	Tamaño del kernel de convolución en *Regional*-GeM.
`gem_p`	*Pooling*	`float`	Potencia utilizada en GeM.
`sgem_ps`	*Pooling*	`float`	Potencia utilizada en *Scale*-GeM.
`sgem_infinity`	*Pooling*	`bool`	Indica si se usa *max-pooling* para *Scale*-GeM.
`eps`	*Pooling*	`float`	Valor mínimo para evitar colapsos numéricos.
`top_k`	Búsqueda	`int`	Número de vecinos a considerar en la búsqueda.
`min_sim`	Búsqueda	`float`	Similitud mínima requerida para aceptar un resultado.
`min_votes`	Búsqueda	`float`	Porcentaje mínimo de votos para validar un resultado.
`remove_inner_boxes`	Búsqueda	`float`	Umbral de solapamiento para descartar cajas contenidas.
`join_boxes`	Búsqueda	`bool`	Indica si se deben unir cajas asociadas al mismo lugar.

Como punto de partida, se emplearon los valores indicados en la Tabla 5.2. Estos valores fueron seleccionados a partir de referencias bibliográficas mencionadas en la Sección 4, de implementaciones oficiales disponibles, o bien a través de aproximaciones informadas cuando no existía una referencia directa.

Bajo esta configuración, se ejecutó el protocolo de evaluación *Revisited Oxford and Paris*, que permite medir el desempeño del sistema en escenarios de recuperación de imágenes.

Tabla 5.2: *Valores default de los parámetros del modelo.*

Parámetro	Valor
image_dim	(640, 640)
allowed_classes	[41, 68, 70, 74, 87, 95, 113, 144, 150, 158, 164, 165, 193, 205, 212, 224, 257, 298, 310, 335, 351, 390, 393, 401, 403, 439, 442, 457, 466, 489, 510, 512, 514, 524, 530, 531, 543, 546, 554, 565, 573, 580, 587, 588, 591]
score_thresh	0.05
iou_thresh	0.25
scales	[0.7071, 1.0, 1.4142]
mean	[0.485, 0.456, 0.406]
std	[0.229, 0.224, 0.225]
rgem_pr	2.50
rgem_size	5
gem_p	4.60
sgem_ps	10.0
sgem_infinity	False
eps	1e-8
topk	5
min_sim	0.80
min_votes	0.55
remove_inner_boxes	0.85
join_boxes	True

5.1.1. Evaluación cuantitativa

La Tabla 5.3 presenta los resultados obtenidos para los conjuntos de datos $\mathcal{R}$Oxford y $\mathcal{R}$Paris. Se reportan las métricas de mAP (*mean Average Precision*) y mP@k (*mean Precision at* k), evaluadas en los tres niveles de dificultad definidos por el protocolo: *easy*, *medium* y *hard*. Además se incorporó el tiempo de inferencia promedio.

Los valores obtenidos evidencian la robustez del modelo frente a variaciones visuales en las imágenes, mostrando un buen rendimiento incluso en escenarios de dificultad elevada, donde las condiciones visuales entre la imagen de consulta y las imágenes de la base difieren significativamente.

5.1.2. Ejemplos cualitativos

La Fig. 5.1 ilustra ejemplos visuales de resultados obtenidos sobre el conjunto $\mathcal{R}$Oxford. En cada fila se muestra, a la izquierda, la imagen de consulta; las columnas siguientes co-

Tabla 5.3: *Resultados en Revisited Oxford and Paris.*

Métrica	$\mathcal{R}$**Oxford**			$\mathcal{R}$**Paris**		
	Easy	*Medium*	*Hard*	*Easy*	*Medium*	*Hard*
mAP	75.49	58.69	30.34	88.19	76.67	55.09
mP@1	97.06	95.71	62.29	97.14	98.57	92.86
mP@5	84.04	81.81	53.43	94.95	97.71	92.00
mP@10	78.46	77.19	44.14	93.38	97.29	89.43
Tiempo		0.7163 s			0.9822 s	

rresponden a las imágenes recuperadas ordenadas por similitud descendente. Las imágenes están etiquetadas visualmente como correctas o incorrectas de acuerdo con el *ground truth* del conjunto.

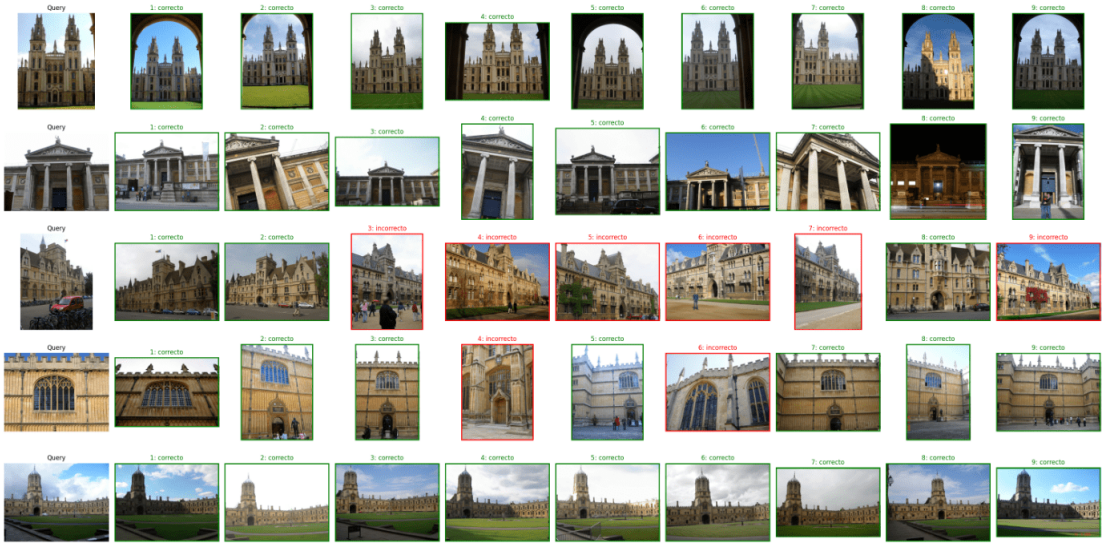


Figura 5.1: *Ejemplos de resultados obtenidos sobre el conjunto $\mathcal{R}$Oxford utilizando imágenes completas. Elaboración propia.*

5.2. Construcción de la base de embeddings

Se desarrolló una función en Python para procesar todas las imágenes de la base de datos y generar sus embeddings utilizando el *pipeline* propuesto (ver Apéndice B). La Fig. 5.2 presenta cuatro estrategias evaluadas para la construcción de dicha base.

Como parte de este proceso, se incorporó un nuevo parámetro denominado `min_sim_db`, el cual establece un umbral mínimo de similitud intraclúster entre imágenes asociadas al mismo punto de interés (*landmark*). Este parámetro permite filtrar las imágenes que no presentan suficiente consistencia visual con el resto del clúster. Además, se analizó el impacto de incluir las detecciones obtenidas dentro de cada imagen (por ejemplo, fachadas o monumentos específicos dentro de una escena más amplia).

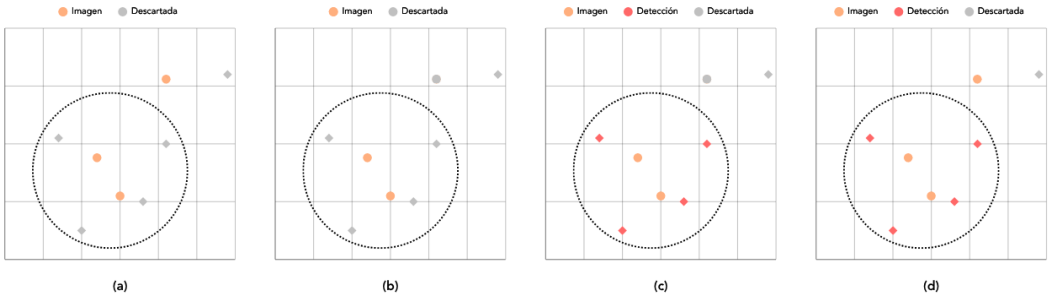


Figura 5.2: *Estrategias para la creación de embeddings.* ***(a)*** *Se consideran todas las imágenes de la base de datos;* ***(b)*** *Se utiliza únicamente el subconjunto de imágenes cuya similitud intraclúster supera el valor de* `min_sim_db`*;* ***(c)*** *Se incluyen tanto las imágenes como sus detecciones, siempre que cumplan el umbral de similitud* `min_sim_db`*;* ***(d)*** *Igual que (c), pero manteniendo todas las imágenes originales, sin aplicar filtrado por similitud. Elaboración propia.*

Se llevaron a cabo múltiples experimentos sobre el conjunto $\mathcal{R}$Oxford, variando el valor del parámetro `min_sim_db` con el objetivo de determinar tanto la estrategia más eficaz como el umbral óptimo de similitud. En la Fig. 5.3 se muestran las curvas de rendimiento obtenidas bajo estas configuraciones.

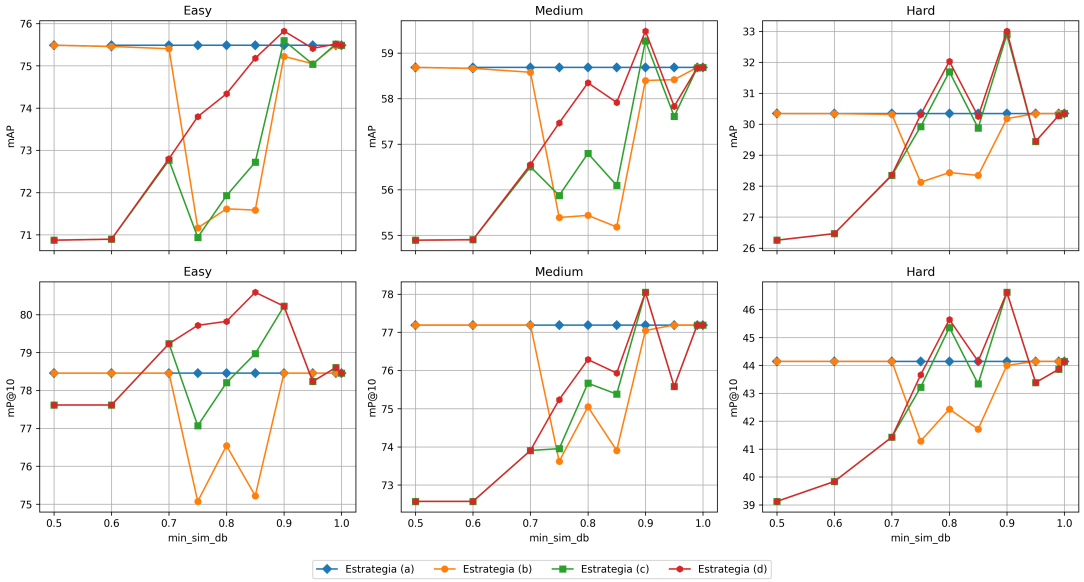


Figura 5.3: *Resultados obtenidos en el conjunto $\mathcal{R}$Oxford al variar el valor de* `min_sim_db` *bajo diferentes estrategias de construcción de la base.*

A partir de las pruebas realizadas, se concluye que la estrategia propuesta en la Fig. 5.2 **(d)**, que combina todas las imágenes completas con sus detecciones siempre que estas presenten una similitud intraclúster mínima de `0.9`, ofrece un rendimiento superior, especialmente en el conjunto de consultas de mayor dificultad (*Hard*).

Por otro lado, la Fig. 5.4 muestra los resultados al modificar el valor de `min_area`, parámetro que establece el área mínima relativa de las *bounding boxes* detectadas en la imagen.

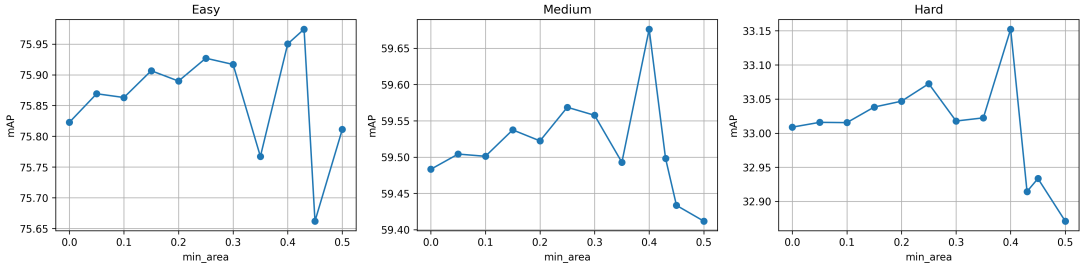


Figura 5.4: *Resultados obtenidos en el conjunto $\mathcal{R}$Oxford al variar el valor de* `min_area`*, que define el área mínima permitida para las detecciones.*

Se observa que un valor de `0.4` mejora el rendimiento global del sistema, incrementando la precisión en todos los niveles de dificultad evaluados.

5.3. Parámetros para la detección

En la Sección 4.2.4.2 se indicó que el modelo utilizado para la detección de objetos fue YOLOv8n, empleando pesos preentrenados sobre el conjunto de datos *Open Images V7*. Este *dataset* contiene un total de 601 clases que abarcan una amplia variedad de dominios.

Para el presente trabajo, se seleccionó un subconjunto de clases con relevancia específica en el contexto turístico, como monumentos, fachadas, estructuras arquitectónicas y elementos patrimoniales. La Tabla 5.4 presenta las clases seleccionadas.

Tabla 5.4: *Subconjunto de clases de Open Images V7.*

ID	Clase	ID	Clase	ID	Clase
41	Bench	224	Gondola	510	Swimming pool
68	Bronze sculpture	257	House	512	Sword
70	Building	298	Ladder	514	Table
74	Bust	310	Lighthouse	524	Teapot
87	Cannon	335	Mirror	531	Tiara
95	Castle	351	Necklace	543	Torch
113	Clock	390	Pitcher (Container)	546	Tower
144	Crown	393	Plant	554	Tree house
150	Dagger	401	Porch	565	Vase
158	Dinosaur	403	Poster	573	Wall clock
164	Door	439	Scoreboard	580	Weapon
165	Door handle	442	Sculpture	587	Window
193	Flag	457	Shotgun	588	Window blind
205	Fountain	466	Skyscraper	591	Wine rack
212	Furniture	489	Stairs		

5.3.1. Filtrado de detecciones

El modelo YOLOv8n genera `8400` *bounding boxes* por imagen, cada una con sus respectivos *scores* asociados a las `601` clases posibles. Estas detecciones se filtran en dos etapas:

- **Filtrado por umbral de *score***: se descartan aquellas cajas cuya puntuación de confianza sea inferior a un valor mínimo definido (`score_thresh`).
- ***Non-Maximum Suppression***: se aplica NMS para eliminar detecciones redundantes, conservando únicamente las cajas con mayor puntuación dentro de regiones solapadas.

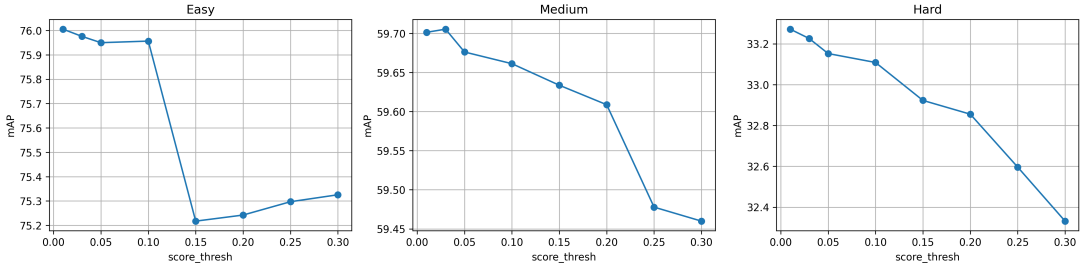


Figura 5.5: *Resultados obtenidos en el conjunto $\mathcal{R}$Oxford al variar el valor de `score_thresh`, que define el score mínimo requerido para conservar una detección.*

La Fig. 5.5 muestra la evolución de la precisión para distintos valores del umbral de *score*. Se observa que, al hacer más restrictivo el filtrado, la precisión tiende a disminuir. Esto se debe a que la identificación del *landmark* se realiza en etapas posteriores al detector, por lo que limitar el número de objetos detectados reduce la cantidad de información disponible para dichas etapas.

El valor de `score_thresh` también afecta el tiempo de inferencia del sistema. Un umbral más bajo incrementa el número de detecciones que deben procesarse, aumentando así el tiempo requerido para completar el *pipeline*. En la Fig. 5.6 se presenta la relación entre el valor de `score_thresh` y el tiempo de inferencia medido sobre el conjunto $\mathcal{R}$Oxford.

Como compromiso entre rendimiento y eficiencia, para la configuración final del sistema se estableció un valor de `score_thresh` igual a `0.05`.

Se aplicó un procedimiento similar para definir el valor del parámetro `iou_thresh`, el cual determina el grado de solapamiento requerido entre dos *bounding boxes* para considerarlas equivalentes y descartar aquella con menor puntuación.

La Fig. 5.7 muestra la variación de la precisión al modificar el valor de `iou_thresh`. A medida que este umbral aumenta, la condición para eliminar cajas solapadas se vuelve más estricta, permitiendo que un mayor número de detecciones pase a las etapas posteriores del sistema. Esta mayor cantidad de información disponible favorece la correcta identificación de los *landmarks*, lo cual se refleja en un incremento de la precisión.

Sin embargo, al igual que en el caso del parámetro `score_thresh`, este aumento en el número de detecciones implica una mayor carga computacional, lo que se traduce en un

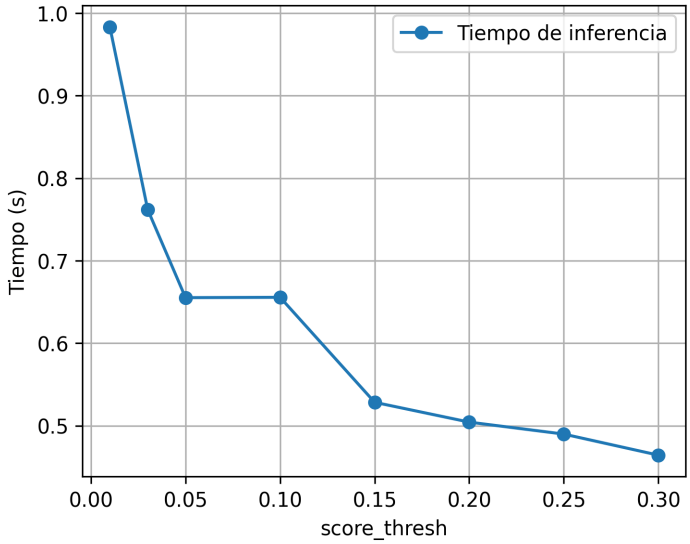


Figura 5.6: *Tiempo de inferencia medido sobre el conjunto $\mathcal{R}$Oxford al variar el valor de* `score_thresh`*.*

incremento del tiempo de inferencia. La Fig. 5.8 ilustra esta relación.

Como compromiso entre precisión y eficiencia, para el resto de este trabajo se fijó el valor de `iou_thresh` en `0.4`.

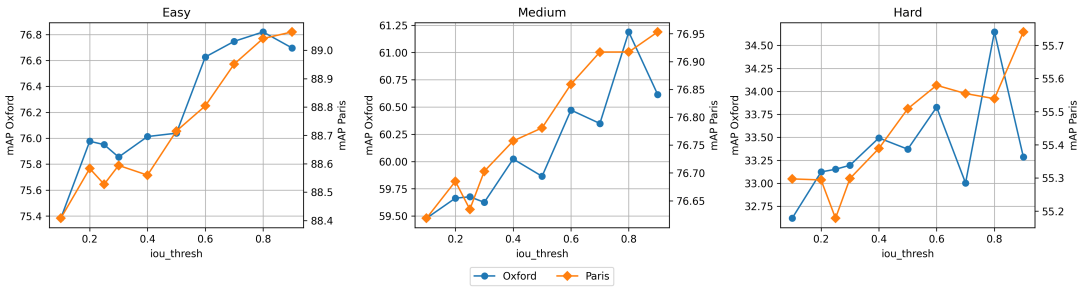


Figura 5.7: *Resultados obtenidos en los conjuntos $\mathcal{R}$Oxford y $\mathcal{R}$Paris al variar el valor de* `iou_thresh`*, que define el solapamiento mínimo requerido para considerar doscajas como redundantes.*

5.4. Parámetros de *SuperGlobal* pooling

En la Sección 3.3.2 se describió que el método *SuperGlobal* se compone de tres etapas consecutivas: *Regional*-GeM, GeM y *Scale*-GeM. Cada una de ellas incorpora parámetros que afectan el cálculo final del descriptor global.

Regional-GeM está controlado por dos hiperparámetros: `rgem_pr`, que corresponde al exponente aplicado en el cálculo del *Generalized-Mean pooling*, y `rgem_size`, que define el tamaño del filtro de *average pooling* aplicado localmente. La Fig. 5.9 muestra el impacto de estos parámetros sobre el rendimiento del sistema en los conjuntos $\mathcal{R}$Oxford y $\mathcal{R}$Paris. El valor óptimo de `rgem_pr` se estableció en `8` y el de `rgem_size` en `5`.

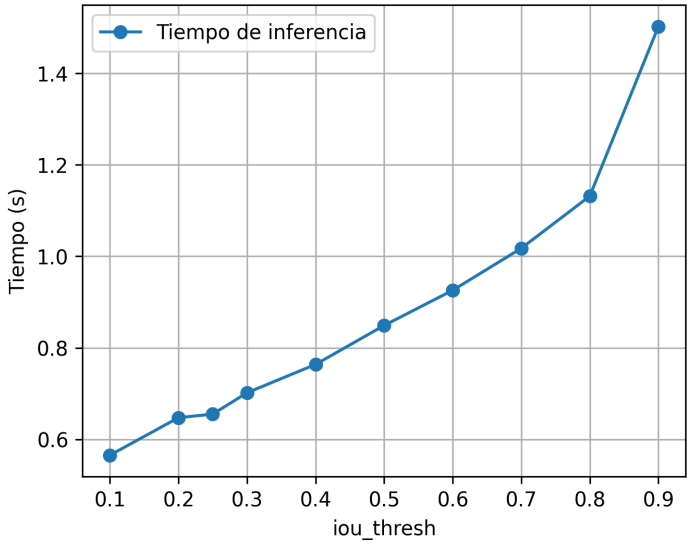


Figura 5.8: *Tiempo de inferencia medido sobre el conjunto $\mathcal{R}$Oxford al variar el valor de* `iou_thresh`*.*

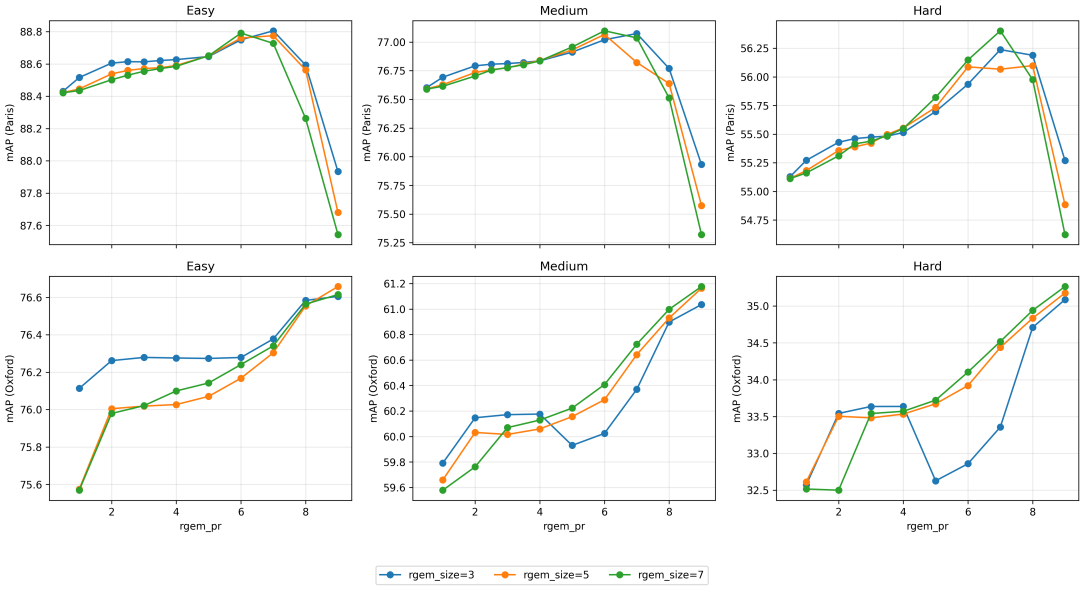


Figura 5.9: *Resultados obtenidos en los conjuntos $\mathcal{R}$Oxford y $\mathcal{R}$Paris al variar los valores de* `rgem_pr` *y* `rgem_size`*.*

A continuación, se aplica una operación de GeM global controlada por el parámetro `gem_p`. La Fig. 5.10 presenta los resultados obtenidos al modificar este parámetro en los conjuntos $\mathcal{R}$Oxford y $\mathcal{R}$Paris. Según los resultados, se seleccionó un valor de `gem_p` igual a `3` para la configuración final.

Finalmente, la etapa de *Scale*-GeM fusiona los descriptores obtenidos a distintas escalas de una misma detección. En la Fig. 5.11 se muestran los resultados para diferentes valores de `sgem_ps`. Además se agregó el valor correspondiente a `sgem_infinity` igual a `True`, que corresponde a aplicar *max-pooling* sobre los *embeddings* de la imágenes escaladas. Se

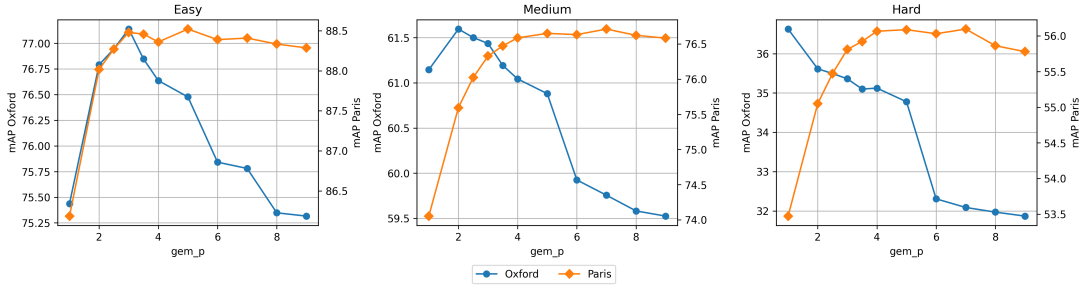


Figura 5.10: *Resultados obtenidos en el dataset $\mathcal{R}$Oxford al variar el valor de* `gem_p`.

determinó este último valor como parte de la configuración final del *pipeline*.

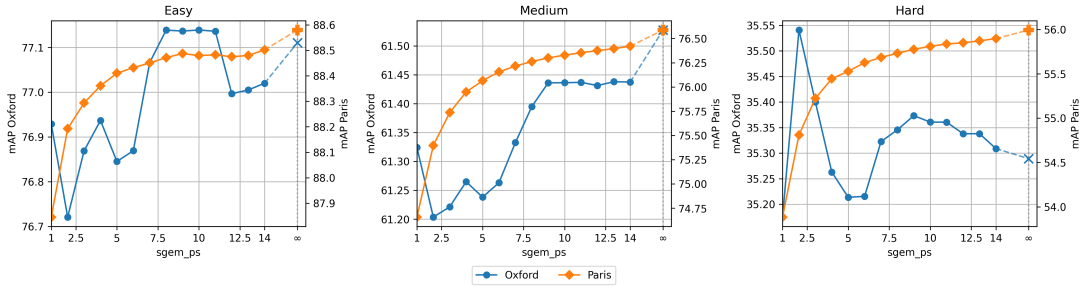


Figura 5.11: *Resultados obtenidos en los datasets $\mathcal{R}$Oxford y $\mathcal{R}$Paris al variar el valor de* `sgem_ps`.

5.5. Resumen de resultados sobre la creación de embeddings

La Tabla 5.5 resume los parámetros seleccionados en este trabajo para la etapa de creación de *embeddings* dentro del *pipeline*. Como resultado del proceso de experimentación, se determinó que la estrategia más eficaz consiste en utilizar todas las imágenes completas, junto con sus respectivas detecciones, siempre que estas cumplan con dos condiciones: mantener una similitud intraclúster superior a un umbral mínimo, y poseer un área relativa mayor que un valor umbral definido.

En la Tabla 5.6 se comparan los valores iniciales —basados en configuraciones reportadas en la bibliografía— con aquellos optimizados durante el desarrollo de este trabajo.

Los resultados muestran mejoras significativas en la precisión sobre el conjunto $\mathcal{R}$Oxford, especialmente en el nivel de dificultad *Hard*. En el conjunto $\mathcal{R}$Paris, las variaciones en precisión fueron más equilibradas, con algunas mejoras y algunas pérdidas según el nivel. Sin embargo, se logró una reducción sustancial del tiempo de inferencia gracias a la optimización del filtrado de detecciones, tal como se detalló en secciones anteriores.

Tabla 5.5: *Parámetros optimizados para la creación de embeddings.*

Parámetro	Valor
min_sim_db	0.90
min_area	0.40
score_thresh	0.05
score_thresh	0.05
iou_thresh	0.40
rgem_pr	8.00
rgem_size	5
gem_p	3.00
sgem_infinity	True

Tabla 5.6: *Comparativa de resultados en Revisited Oxford and Paris.*

Métrica	$\mathcal{R}$Oxford			$\mathcal{R}$Paris		
	Easy	*Medium*	*Hard*	*Easy*	*Medium*	*Hard*
mAP	77.11 (+2.1 %)	61.52 (+4.8 %)	35.29 (+16.31 %)	88.58 (+0.4 %)	76.59 (-0.1 %)	55.99 (+1.6 %)
mP@1	97.06 (+0.0 %)	95.71 (+0.0 %)	74.29 (+19.26 %)	98.57 (+1.5 %)	98.57 (+0.0 %)	91.42 (-1.5 %)
mP@5	86.84 (+3.3 %)	85.86 (+5.0 %)	59.43 (+11.2 %)	95.52 (+0.6 %)	97.43 (-0.3 %)	91.43 (-0.6 %)
mP@10	83.01 (+5.8 %)	80.71 (+4.6 %)	48.71 (+10.4 %)	93.52 (+0.1 %)	96.57 (-0.7 %)	89.00 (-0.5 %)
Tiempo	0.7320 s (+2.2 %)			0.8229 s (-16.2 %)		

5.6. Parámetros de búsqueda

Una vez generados los *embeddings* tanto para las imágenes de la base de datos como para las imágenes de consulta (*queries*), es posible calcular la similitud entre ellos. Dado que los vectores se encuentran previamente normalizados con norma L2, la medida de similitud coseno se reduce al producto escalar.

A partir de estos valores, se ordenan los resultados de forma descendente según su similitud, obteniendo así un ranking de imágenes de la base más cercanas a cada *query*. Al reemplazar las identificaciones individuales de las imágenes por los *landmarks* a los que pertenecen, se construye un ranking de *landmarks* por consulta.

5.6.1. Selección del *landmark*

Una vez generado el ranking de *landmarks* para cada imagen de consulta, el siguiente paso consiste en seleccionar el *landmark* predicho utilizando un criterio de votación sobre los elementos más relevantes del ranking.

Este proceso está controlado por dos parámetros clave:

- `top_k`: define la cantidad de resultados considerados para la votación.
- `min_votes`: establece la cantidad mínima de votos que un *landmark* debe recibir para ser seleccionado.

Ambos parámetros están interrelacionados y deben analizarse de manera conjunta. Un valor bajo de `top_k` reduce el costo computacional y el tamaño de la matriz de similitud, pero puede limitar la información disponible en casos complejos. Por el contrario, un valor excesivamente alto puede incluir resultados irrelevantes e incluso superar el número real de imágenes asociadas a un *landmark*, lo que puede introducir ruido. `top_k` tiene un impacto directo sobre la precisión del sistema.

El parámetro `min_votes`, por su parte, regula el nivel de confianza requerido para aceptar una predicción. Al exigir una cantidad mínima de votos, se reducen los falsos positivos, pero también puede disminuir la cobertura del sistema al dejar sin respuesta aquellas consultas donde no se alcanza dicho umbral.

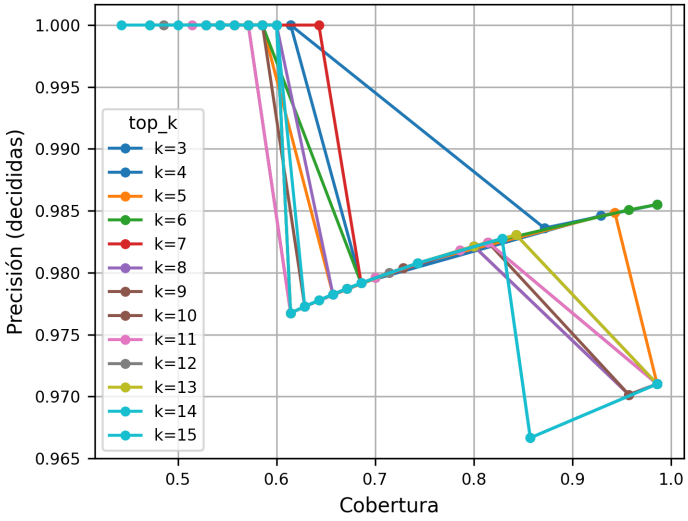


Figura 5.12: *Relación entre precisión y cobertura sobre el dataset $\mathcal{R}$Oxford para diferentes valores de `top_k`.*

La Fig. 5.12 muestra la relación entre precisión y cobertura al variar `top_k`. Se observa que un valor de `top_k` igual a 6 permite alcanzar un equilibrio óptimo entre ambas métricas para algún valor de `min_votes`.

En la Fig. 5.13 se analiza el impacto de `min_votes` manteniendo fijo `top_k` en 6. Los resultados indican que el valor óptimo de `min_votes` es 1. Esto implica que si no existe un *landmark* con múltiples coincidencias dentro de los seis primeros resultados, se seleccionará el que posea mayor similitud con la *query*. En cambio, si algún *landmark* aparece más de una vez, se le asignará prioridad en la selección, incluso si no ocupa la primera posición del ranking.

Podría ocurrir que dentro del conjunto de resultados definido por `top_k` existan imágenes con una similitud muy baja respecto a la imagen de consulta. En estos casos, permitir que dichas imágenes participen en la votación por mayoría puede derivar en la selección de un *landmark* incorrecto.

Para mitigar este efecto, se incorporó un parámetro adicional, `min_sim`, que establece un umbral mínimo de similitud requerido para que un resultado sea considerado en la votación.

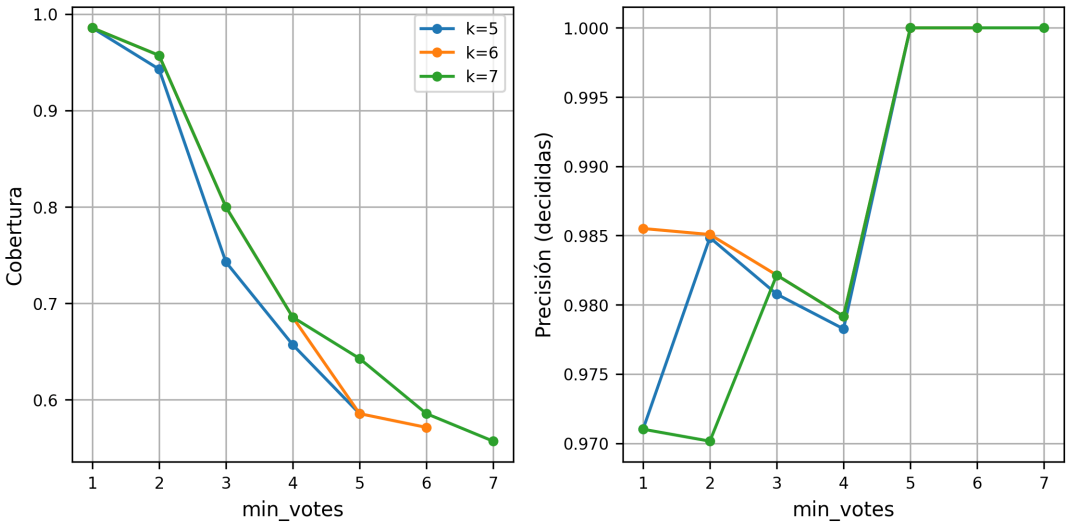


Figura 5.13: *Precisión y cobertura en función de* `min_votes`.

De este modo, sólo los resultados suficientemente similares a la *query* influyen en la decisión final.

La Fig. 5.14 muestra cómo varían la precisión y la cobertura al modificar el valor de `min_sim` en el conjunto $\mathcal{R}$Oxford. A partir de este análisis, se determinó que un valor de `0.84` proporciona un buen equilibrio entre ambas métricas, por lo que fue adoptado en la configuración final del *pipeline*.

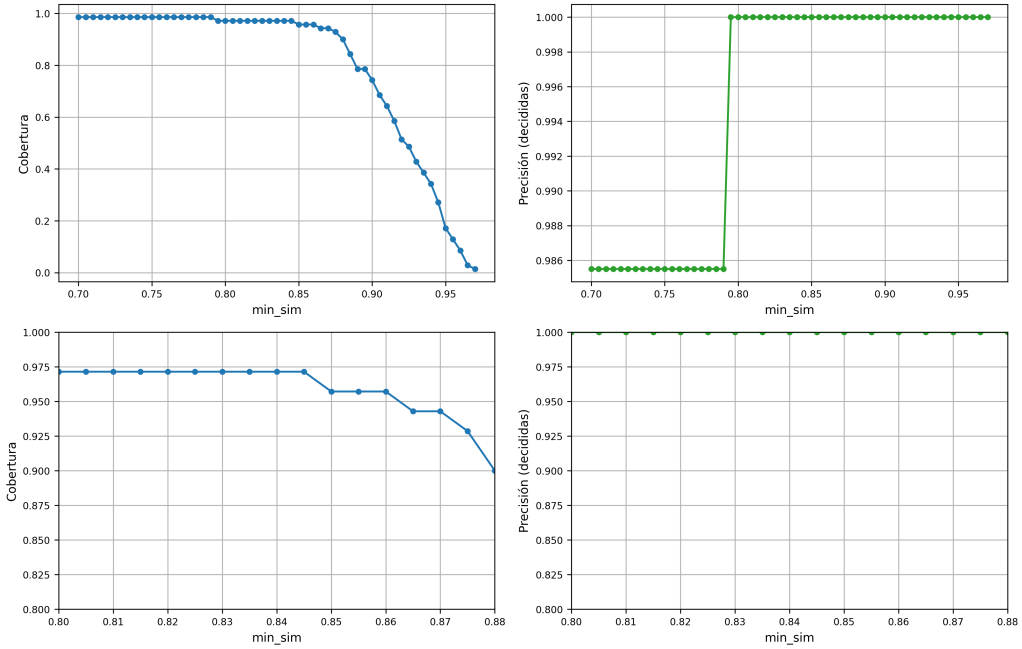


Figura 5.14: *Precisión y cobertura en función de* `min_sim` *sobre el dataset* $\mathcal{R}$Oxford.

5.6.2. Refinamiento del resultado

En esta etapa final del *pipeline*, se dispone de un conjunto de *bounding boxes* asociadas a un *landmark* identificado como relevante. El objetivo del postprocesamiento es refinar la localización de dichos *landmarks* en la imagen, mejorando la calidad espacial de las detecciones presentadas como salida.

En primer lugar, se eliminan las detecciones redundantes que corresponden al mismo *landmark* y cuyas cajas están contenidas unas dentro de otras. Este procedimiento es similar al de la *Non-Maximum Suppression*, pero en lugar de conservar la detección con mayor *score*, se mantiene aquella con menor área, bajo la hipótesis de que las cajas más pequeñas son más precisas cuando existe solapamiento completo. En la Fig. 5.15 se muestra un ejemplo: la parte **(a)** ilustra múltiples detecciones superpuestas del mismo *landmark*, mientras que la parte **(b)** muestra el resultado tras aplicar el filtrado.



Figura 5.15: *Ejemplo de eliminación de detecciones redundantes.* ***(a)*** *Imagen con múltiples detecciones superpuestas de un mismo landmark.* ***(b)*** *Resultado tras eliminar las cajas contenidas. Elaboración propia.*

Por último, si el resultado final incluye varias detecciones correspondientes al mismo *landmark*, pero no contenidas entre sí, se agrupan en una única *bounding box* que las englobe. Este paso permite consolidar la predicción y facilitar su interpretación. La Fig. 5.16 ilustra este proceso.

5.7. Resultados sobre Cáceres

Con los parámetros definidos en las secciones anteriores, se procedió a exportar el *pipeline* al formato ONNX, generando un archivo de `105.763 KB`. La base de datos se construyó

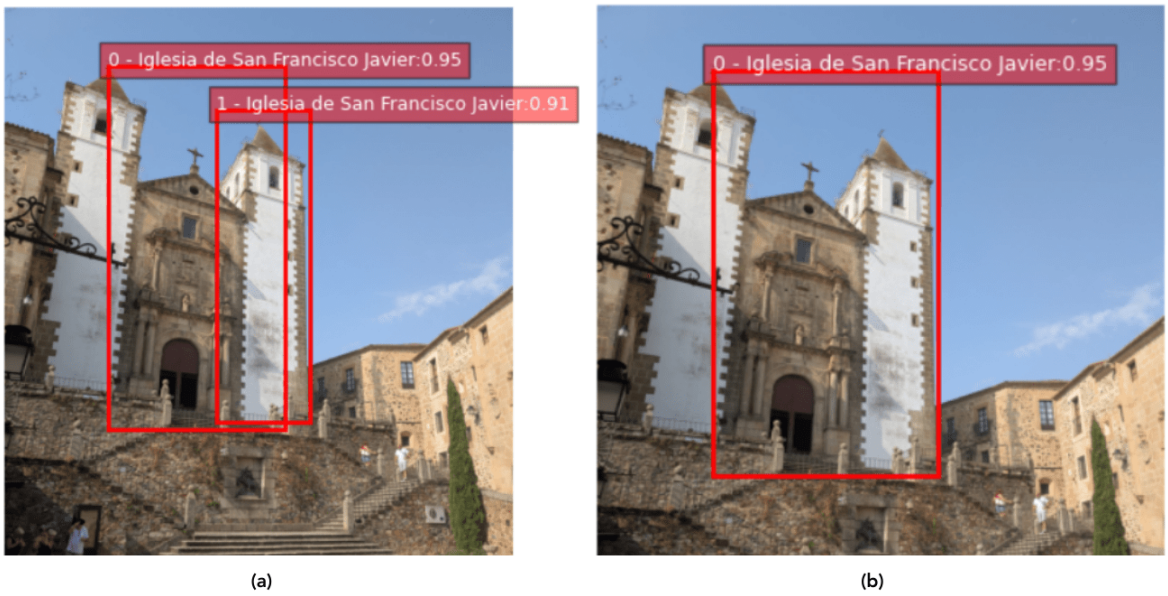


Figura 5.16: *Ejemplo de agrupación de detecciones del mismo landmark.* ***(a)*** *Imagen con múltiples detecciones separadas.* ***(b)*** *Resultado tras agruparlas en una única caja envolvente. Elaboración propia.*

con un total de 489 imágenes correspondientes a 14 puntos de interés turístico de la ciudad de Cáceres, España. A partir de estas imágenes, se generaron y almacenaron los *embeddings* en un archivo binario de `7.564 KB`.

En la Fig. 5.17 se presentan ejemplos en los que el sistema logró identificar correctamente los *landmarks*, incluso bajo variaciones significativas en el punto de vista o en las condiciones de iluminación.

En la Fig. 5.18 se analiza un caso particular con dos edificios vecinos. En **(a)** se observa que, cuando ambos edificios aparecen claramente separados en la imagen, el sistema es capaz de detectarlos e identificarlos de manera independiente. Sin embargo, cuando la imagen se toma desde un ángulo donde ambos se solapan, sólo uno de ellos es detectado. En **(b)** se muestran las *bounding boxes* generadas por el detector, lo que confirma que en dicha situación el sistema no logra detectar los edificios de forma separada, afectando la identificación final.

Finalmente, la Fig. 5.19 muestra ejemplos en los que ciertos *landmarks* presentes en las imágenes no fueron detectados por el sistema. Esto puede deberse a diversos factores, como oclusión parcial, baja resolución del objeto de interés o diferencias significativas respecto a las imágenes de la base de datos.



Figura 5.17: *Ejemplos con detecciones correctas sobre Cáceres. Elaboración propia.*



Figura 5.18: *Ejemplo de detección parcial en sitios vecinos.* ***(a)*** *Salida del sistema para distintas imágenes tomadas desde ángulos diversos: cuando los edificios aparecen claramente separados, ambos son identificados correctamente; si se presentan juntos, solo uno es reconocido.* ***(b)*** *Salida del detector de objetos sobre las mismas imágenes, donde se observa que en casos de solapamiento los edificios son agrupados en una única detección.*



Figura 5.19: *Ejemplos de casos no detectados sobre Cáceres. Elaboración propia.*

Conclusiones, limitaciones y perspectivas a futuro

6

En esta sección se presentan las conclusiones finales del trabajo, las limitaciones identificadas y las perspectivas de mejora y continuidad en investigaciones futuras.

6.1. Conclusiones

A continuación se presentan las conclusiones del trabajo, organizadas en correspondencia con los objetivos planteados en la Sección 1.3:

1. **Analizar el estado del arte en detección de objetos y técnicas de *image retrieval*.** Se realizó una revisión exhaustiva de los principales enfoques de detección de objetos, desde las arquitecturas basadas en regiones como Faster R-CNN hasta los métodos de una sola etapa como SSD y YOLO, incluyendo sus evoluciones más recientes. En paralelo, se estudiaron las técnicas de *instance image retrieval*, abarcando tanto los métodos clásicos basados en descriptores manuales (SIFT, SURF, BoW) como los enfoques modernos basados en aprendizaje profundo (GeM, NetVLAD, DELG, CVNet y SuperGlobal). Este análisis permitió fundamentar teóricamente las decisiones de diseño del *pipeline* desarrollado.

2. **Implementar una arquitectura de redes neuronales profundas robusta frente a variaciones visuales, oclusiones y elementos distractores.** Se diseñó e implementó un *pipeline* modular de cinco etapas que combina detección de objetos mediante YOLOv8n, extracción de características con ResNet50 entrenada bajo el esquema CVNet-Global y un mecanismo de *pooling* avanzado basado en SuperGlobal. La arquitectura propuesta demostró ser robusta ante cambios de iluminación, escala, ángulo de captura y presencia de elementos no relevantes, logrando descriptores discriminativos y compactos para la identificación de lugares.

3. **Diseñar una base de datos de puntos de interés turístico estructurada mediante vectores de características visuales.** Se construyó una base de datos de *embeddings* visuales a partir de imágenes turísticas de la ciudad de Cáceres. La base se generó mediante el *pipeline* propuesto y se almacenó en forma de un *DataFrame* y una matriz de descriptores normalizados L2. Se comprobó que los *embeddings* poseen un tamaño

reducido, lo que facilita su descarga y uso en dispositivos móviles sin requerir conexión a internet, garantizando la viabilidad práctica de la solución.

4. **Integrar el sistema con un *framework* de realidad aumentada en dispositivos móviles.** El modelo completo fue exportado al formato ONNX e integrado en una aplicación Android mediante ONNX Runtime. El sistema procesa en tiempo real la imagen capturada por la cámara y devuelve las detecciones junto con la identificación del *landmark*, que se muestran superpuestas sobre la escena a través de ARCore. De este modo, se consiguió una interacción fluida y una experiencia inmersiva que valida la integración del sistema con un entorno de realidad aumentada.

5. **Evaluar el desempeño del sistema utilizando métricas de precisión y tiempo de inferencia.** Se evaluó el sistema en los *benchmarks Revisited Oxford and Paris*, obteniendo métricas de mAP y mP@k competitivas incluso en escenarios de alta dificultad. Además, se midieron los tiempos de inferencia, situados en torno a los 800 ms por imagen, lo que si bien no corresponde a tiempo real estricto, resulta suficiente en el contexto turístico dado que los *landmarks* son estructuras estáticas y la estabilidad visual puede ser garantizada por los anclajes de RA. Estas evaluaciones permiten concluir que el sistema ofrece un equilibrio adecuado entre precisión, cobertura y eficiencia en un entorno móvil.

6.2. Limitaciones

El trabajo desarrollado presenta algunas limitaciones relevantes:

a. **Backbones separados para detección y extracción.** El sistema utiliza YOLOv8n para detección y ResNet50 (CVNet-Global) para extracción, lo que implica procesar la imagen dos veces y reduce la eficiencia. Una arquitectura unificada permitiría optimizar recursos.

b. **Carencia de datasets con anotaciones conjuntas.** Actualmente no existe un dataset lo suficientemente amplio que combine *landmarks* con anotaciones de *bounding boxes*. Esto obligó a emplear Open Images V7 para la detección y Google Landmarks v2 para la extracción, limitando la posibilidad de entrenar un modelo *end-to-end*.

c. **Escalabilidad de la base de datos.** Aunque los *embeddings* son compactos, un dataset global con millones de lugares generaría una base demasiado grande para dispositivos móviles, comprometiendo tanto almacenamiento como eficiencia de búsqueda.

d. **Dependencia del detector.** El sistema presenta una fuerte dependencia de la etapa de detección: si un objeto relevante no es identificado por el detector, no podrá ser procesado por las etapas posteriores. Este problema se mitiga parcialmente al incluir

siempre la imagen completa como una detección adicional; sin embargo, en escenarios con múltiples *landmarks* en la misma escena, si el detector no logra diferenciarlos de manera correcta, dichos lugares no podrán ser reconocidos por el sistema.

6.3. Perspectivas a futuro

El presente trabajo constituye una primera aproximación hacia sistemas móviles de reconocimiento de lugares con integración en realidad aumentada. Entre las principales líneas de mejora se destacan:

a. **Creación de un dataset especializado.** Construir un conjunto de datos a gran escala que incluya múltiples *landmarks* por imagen con anotaciones de *bounding boxes*, distribuido geográficamente, para entrenar modelos *end-to-end* más eficientes.

b. **Optimización de la búsqueda.** Incorporar técnicas de indexación como FAISS o HNSW y filtrar candidatos mediante datos de GPS o ubicación aproximada, reduciendo el espacio de búsqueda y mejorando la latencia.

c. **Refinamiento contextual.** Explorar métodos de *reranking* como CVNet-Rerank o el uso de probabilidades condicionales cuando coexisten múltiples *landmarks* en la misma imagen, mejorando la fiabilidad de las predicciones.

d. **Extensión hacia guías turísticas inteligentes.** Integrar un módulo adicional que brinde información contextual (histórica, cultural o práctica) sobre los lugares detectados, en un formato interactivo y atractivo para el usuario final.

e. **Migración a nuevos dispositivos de RA.** Adaptar el sistema a gafas inteligentes y visores de realidad aumentada, donde la latencia y el consumo energético son aún más críticos, pero representan el futuro natural de estas aplicaciones.

Lista de Acrónimos

CNN *Convolutional Neural Network*.

CRISP-DM *Cross-Industry Standard Process for Data Mining*.

CVNet *Correlation Verification Networks*.

FPN *Feature Pyramid Networks*.

FPS Fotogramas por segundo *Frames Per Second*.

GeM *Generalized-Mean pooling*.

IoU *Intersection over Union*.

NMS *Non-Maximum Suppression*.

ONNX *Open Neural Network Exchange*.

RA Realidad Aumentada.

R-CNN *Region-based CNN*.

ResNet *Residual Network*.

RPN *Region Proposal Network*.

SSD *Single Shot Detector*.

VPR *Visual Place Recognition*.

YOLO *You Only Look Once*.

Apéndice A



Todo el código desarrollado para este proyecto se encuentra disponible en el repositorio de GitHub, accesible en el siguiente enlace:

https://github.com/theviderlab/landmark-detection-with-retrieval

El repositorio está organizado en las siguientes carpetas principales:

- `landmark_detection`: implementación completa del *pipeline*, incluyendo todas sus etapas. La clase `Pipeline_Landmark_Detection` centraliza la configuración del sistema: recibe los parámetros, instancia cada módulo, permite exportar a formato ONNX y ejecutar etapas de manera individual cuando se requiera.
- `datasets`: contiene los conjuntos de datos utilizados en la evaluación, en particular el protocolo *Revisited Oxford and Paris*, junto con un conjunto personalizado de imágenes de la ciudad de Cáceres.
- `benchmark`: incluye el código oficial del protocolo *Revisited Oxford and Paris*, extendido con funciones adicionales que permiten evaluar el rendimiento del *pipeline* bajo diferentes configuraciones.
- `android_app`: código fuente de la aplicación móvil desarrollada en Android Studio, utilizada para demostrar la viabilidad de integrar el sistema en un entorno de realidad aumentada.

Adicionalmente, el repositorio contiene diversos *notebooks* empleados durante el desarrollo, utilizados para experimentación, análisis intermedio y visualización de resultados.

Apéndice B

B

El sistema ha sido diseñado de manera que la incorporación de nuevas imágenes a la base de datos resulte sencilla y eficiente. Para ello, la clase `Pipeline_Landmark_Detection` dispone del método `build_image_database`, que recibe como entrada la ruta a una carpeta con las imágenes a procesar, así como las rutas de los ficheros de salida donde se almacenarán los resultados. Adicionalmente, se debe proporcionar un *dataframe* con el *ground truth* correspondiente a cada imagen.

La función guarda los datos procesados de forma periódica en el disco, evitando reprocesar imágenes ya analizadas en ejecuciones anteriores, salvo que se indique lo contrario mediante el parámetro `force_rebuild`. Asimismo, admite los tres parámetros de configuración descritos en este trabajo: `min_area`, `min_sim` y `keep_full_img`.

Un ejemplo de invocación del método es el siguiente:

```
df, descriptors, places_db = pipeline.build_image_database(
    image_folder = IMAGES_PATH,
    df_pickle_path = DATABASE_DF,
    descriptor_pickle_path = DATABASE_DESC,
    return_places_db = True,
    force_rebuild = False,
    save_every = 500,
    min_area = 0.1,
    min_sim = 0.8,
    keep_full_img = True
)
```

Bibliografía

Arandjelović, R., Gronat, P., Torii, A., Pajdla, T., y Sivic, J. (2016). NetVLAD: CNN architecture for weakly supervised place recognition. 2459.

Babenko, A. y Lempitsky, V. (2015). Aggregating Deep Convolutional Features for Image Retrieval. volume 676. arXiv.

Bay, H., Tuytelaars, T., y Van Gool, L. (2006). SURF: Speeded Up Robust Features. In Leonardis, A., Bischof, H., y Pinz, A., editors, *Computer Vision – ECCV 2006*, volume 9654, pages 404–417, Berlin, Heidelberg. Springer Berlin Heidelberg.

Bochkovskiy, A., Wang, C.-Y., y Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection.

Cadena, C., Galvez-López, D., Tardos, J. D., y Neira, J. (2012). Robust Place Recognition With Stereo Sequences. *IEEE Transactions on Robotics*, 85(4):871–885.

Cao, B., Araujo, A., y Sim, J. (2020). Unifying Deep Local and Global Features for Image Search. In Vedaldi, A., Bischof, H., Brox, T., y Frahm, J.-M., editors, *Computer Vision – ECCV 2020*, volume 285, pages 726–743. Springer International Publishing, Cham.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., y Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide.

Chen, Z., Lam, O., Jacobson, A., y Milford, M. (2014). Convolutional Neural Network-based Place Recognition. volume 260. arXiv.

Cummins, M. y Newman, P. (2011). Appearance-only SLAM at large scale with FAB-MAP 2.0. *The International Journal of Robotics Research*, 681(9):1100–1123.

Girshick, R. (2015). Fast R-CNN. volume 24144. arXiv.

Girshick, R., Donahue, J., Darrell, T., y Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. volume 25446. arXiv.

He, K., Gkioxari, G., Dollár, P., y Girshick, R. (2018). Mask R-CNN. volume 25409. arXiv.

He, K., Zhang, X., Ren, S., y Sun, J. (2015). Deep Residual Learning for Image Recognition. volume 182466. arXiv.

Hou, Y., Zhang, H., y Zhou, S. (2015). Convolutional Neural Network-Based Image Representation for Visual Loop Closure Detection. volume 162. arXiv.

Huang, Y., Wang, Y., Tai, Y., Liu, X., Shen, P., Li, S., Li, J., y Huang, F. (2020). CurricularFace: Adaptive Curriculum Learning Loss for Deep Face Recognition. arXiv.

Kalantidis, Y., Mellina, C., y Osindero, S. (2016). Cross-dimensional Weighting for Aggregated Deep Convolutional Features. 425.

Khanam, R. y Hussain, M. (2024). YOLOv11: An Overview of the Key Architectural Enhancements.

Krasin, I., Duerig, T., Alldrin, N., Ferrari, V., Abu-El-Haija, S., Kuznetsova, A., Rom, H., Uijlings, J., Popov, S., Kamali, S., Malloci, M., Pont-Tuset, J., Veit, A., Belongie, S., Gomes, V., Gupta, A., Sun, C., Chechik, G., Cai, D., Feng, Z., Narayanan, D., y Murphy, K. (2017). OpenImages: A public dataset for large-scale multi-label and multi-class image classification. *Dataset available from https://storage.googleapis.com/openimages/web/index.html*.

Kröse, B., Vlassis, N., Bunschoten, R., y Motomura, Y. (2001). A probabilistic model for appearance-based robot localization. *Image and Vision Computing*, 172(6):381–391.

Lamon, P., Nourbakhsh, I., Jensen, B., y Siegwart, R. (2001). Deriving and matching image fingerprint sequences for mobile robot localization. In *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*, volume 98, pages 1609–1614, Seoul, South Korea. IEEE.

Lee, S., Seong, H., Lee, S., y Kim, E. (2022). Correlation Verification for Image Retrieval. 39.

Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., y Wei, X. (2022). YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications.

Liu, L., Ouyang, W., Wang, X., Fieguth, P., Chen, J., Liu, X., y Pietikäinen, M. (2019). Deep Learning for Generic Object Detection: A Survey. volume 2379. arXiv.

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., y Berg, A. C. (2016). SSD: Single Shot MultiBox Detector. volume 28692, pages 21–37.

Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 46480(2):91–110.

Martin A. Fischler, Fischler, M. A., Martin A. Fischler, Robert C. Bolles, y Bolles, R. C. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of The ACM*, 24(6):381–395.

Martinez-Plumed, F., Contreras-Ochando, L., Ferri, C., Hernandez-Orallo, J., Kull, M., Lachiche, N., Ramirez-Quintana, M. J., y Flach, P. (2021). CRISP-DM Twenty Years Later: From Data Mining Processes to Data Science Trajectories. *IEEE Transactions on Knowledge and Data Engineering*, 33(8):3048–3061.

Noh, H., Araujo, A., Sim, J., Weyand, T., y Han, B. (2018). Large-Scale Image Retrieval with Attentive Deep Local Features. 736.

Oliva, A. y Torralba, A. (2006). Building the gist of a scene: The role of global image features in recognition. In *Progress in Brain Research*, volume 1514, pages 23–36. Elsevier.

Radenović, F., Iscen, A., Tolias, G., Avrithis, Y., y Chum, O. (2018a). Revisiting Oxford and Paris: Large-Scale Image Retrieval Benchmarking. 346.

Radenović, F., Tolias, G., y Chum, O. (2018b). Fine-tuning CNN Image Retrieval with No Human Annotation. 1198.

Redmon, J., Divvala, S., Girshick, R., y Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. *arXiv.org*, 36736.

Redmon, J. y Farhadi, A. (2016). YOLO9000: Better, Faster, Stronger.

Redmon, J. y Farhadi, A. (2018). YOLOv3: An Incremental Improvement.

Ren, S., He, K., Girshick, R., y Sun, J. (2016). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. volume 58610. arXiv.

Sarlin, P.-E., Cadena, C., Siegwart, R., y Dymczyk, M. (2019). From Coarse to Fine: Robust Hierarchical Localization at Large Scale. 775.

Sermanet, P., Eigen, D., Zhang, X., Mathieu, M., Fergus, R., y LeCun, Y. (2014). OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks. volume 4941. arXiv.

Shao, S., Chen, K., Karpur, A., Cui, Q., Araujo, A., y Cao, B. (2023). Global Features are All You Need for Image Retrieval and Reranking. 19.

Sivic y Zisserman (2003). Video Google: A text retrieval approach to object matching in videos. In *Proceedings Ninth IEEE International Conference on Computer Vision*, volume 7075, pages 1470–1477 vol.2, Nice, France. IEEE.

Sünderhauf, N., Dayoub, F., Shirazi, S., Upcroft, B., y Milford, M. (2015). On the Performance of ConvNet Features for Place Recognition. volume 524. arXiv.

Taira, H., Okutomi, M., Sattler, T., Cimpoi, M., Pollefeys, M., Sivic, J., Pajdla, T., y Torii, A. (2021). InLoc: Indoor Visual Localization with Dense Matching and View Synthesis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 439(4):1293–1307.

Tian, Y., Ye, Q., y Doermann, D. (2025). YOLOv12: Attention-Centric Real-Time Object Detectors. https://arxiv.org/abs/2502.12524v1.

Tolias, G., Sicre, R., y Jégou, H. (2016). Particular object retrieval with integral max-pooling of CNN activations. 943.

Torii, A., Sivic, J., Okutomi, M., y Pajdla, T. (2015). Visual Place Recognition with Repetitive Structures. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 386(11):2346–2359.

Turcot, P. y Lowe, D. G. (2009). Better matching with fewer features: The selection of useful features in large database recognition problems. In *2009 IEEE 12th International Conference on Computer Vision Workshops, ICCV Workshops*, volume 268, pages 2109–2116, Kyoto, Japan. IEEE.

Ulrich, I. y Nourbakhsh, I. (2000). Appearance-based place recognition for topological localization. In *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, volume 636, pages 1023–1029, San Francisco, CA, USA. IEEE.

Valgren, C. y Lilienthal, A. J. (2010). SIFT, SURF & seasons: Appearance-based long-term localization in outdoor environments. *Robotics and Autonomous Systems*, 229(2):149–156.

Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., y Ding, G. (2024a). YOLOv10: Real-Time End-to-End Object Detection.

Wang, C.-Y., Bochkovskiy, A., y Liao, H.-Y. M. (2022). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors.

Wang, C.-Y., Yeh, I.-H., y Liao, H.-Y. M. (2024b). YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information.

Weyand, T., Araujo, A., Cao, B., y Sim, J. (2020). Google Landmarks Dataset v2 – A Large-Scale Benchmark for Instance-Level Recognition and Retrieval. 322.

Yaseen, M. (2024). What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector.

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., y Chen, J. (2024). DETRs Beat YOLOs on Real-time Object Detection. volume 495. arXiv.